UNTOLD EDUCATION

Untold skills for modern jobs

CSS
Complete Notes

Styling modern websites from basics to advanced

color: #2196f3;

```
<htinl>
color:
  #2196f3;
margin: 0 auto;
display: flex;
padding: 20px;
```

grid {

Created by **Pawan Soni**

[redacted] • Rewari, Haryana

Contents

✅ What is CSS? (CSS kya hota hai?) 4

✅ History of CSS (CSS Ka Itihaas) 4

✅ CSS Ke Various Versions / Levels – Timeline Table 4

✅ CSS Syntax (CSS ka basic likhne ka tarika) 5

✅ CSS Comments (CSS mein comment kaise likhte hain?) 5

✅ CSS TEXT PROPERTIES – 7

✅ CSS FONT PROPERTIES – 11

✅ CSS UNITS – 14

- 🔷 1. ABSOLUTE UNITS 14
- 🔷 2. RELATIVE UNITS 16

✅ CSS BACKGROUND – 18

✅ CSS BORDER – 23

✅ CSS MARGIN –

..... 28

✅ CSS PADDING – 31

✅ CSS BOX MODEL – 34

✅ CSS SELECTORS – 39

✅ CSS POSITION – 44

✅ CSS z-index – 49

📌 CSS GRADIENT 53

BOX SHADOWS IN CSS 59

✅ ALL CSS FILTER FUNCTIONS 62

📋 LISTS IN CSS — 67

ANCHOR STATES IN CSS — 74

COMBINATORS IN CSS — 79

📦 **DISPLAY IN CSS — s** 82
🌊 **OVERFLOW IN CSS —** 84
✨ **PSEUDO-ELEMENTS IN CSS —** 89
🎯 **PSEUDO-CLASSES IN CSS —** 95
📰 **COLUMN LAYOUT IN CSS —** 99
🧩 **FLEXBOX IN CSS —** 106
🧱 **GRID IN CSS —** 112
🔄 **TRANSFORM IN CSS —** 119
🎞 **TRANSITIONS IN CSS —** 127
🎬 **ANIMATIONS IN CSS —** 133

UNTOLD EDUCATION
Untold skills for modern jobs
CSS
Complete Notes
Styling modern websites from basics to advanced
Created by **Pawan Soni**
Computer Teacher · Rewari, Haryana

✅ What is CSS? (CSS kya hota hai?)

CSS ka full form hai **Cascading Style Sheets**. Ye woh language hai jo website ko **sundar banati hai** aur **design karti hai**.
HTML → sirf structure deta hai (jaise headings, paragraphs, buttons, images etc.) CSS → us structure ko **color, size, spacing, position, animation** sab kuch deta hai.
Example socho: HTML = ghar ki deewar, darwaza, khidki (sirf structure) CSS = paint, tiles, furniture, lighting, decoration (sundarta)

✅ History of CSS (CSS Ka Itihaas)

- **Idea kab aaya?** 1994 mein **Håkon Wium Lie** (CERN mein Tim Berners-Lee ke saath kaam kar rahe the) ne pehli baar **Cascading Style Sheets** ka idea propose kiya. Isse pehle websites mein design HTML ke andar hi likha jaata tha (jaise <font color="red">), jo bahut messy ho jata tha. CSS ne content (HTML) aur design ko alag kar diya – ye bahut badi revolution thi!
- **Pehla official version kab aaya?** 17 December 1996 → **CSS Level 1** (CSS1) W3C Recommendation bana. Isne basic styling jaise font, color, margin, padding introduce kiya.
- **Uske baad kya hua?** Browser companies (Netscape, Internet Explorer) ne CSS support shuru kiya, lekin bahut bugs the. Fir improvements aate gaye.
- **Aaj ka scene (2026 tak)** CSS ab **single version** nahi hai. **CSS3** ke baad se ye **modules** mein divide ho gaya hai. Har module alag-alag develop hota hai (jaise Flexbox, Grid, Animations, Colors). Isliye hum bolte hain "CSS Level 3 and beyond" ya bas "modern CSS".

✅ CSS Ke Various Versions / Levels – Timeline Table

Version / Level	Release Date	Kya Naya Aaya? (Main Features)	Status / Notes
CSS Level 1 (CSS1)	December 1996	Font properties, colors, text alignment, margins, padding, borders, background	Pehla official standard – basic styling
CSS Level 2 (CSS2)	May 1998	Positioning (absolute, relative, fixed), z-index, media types, bidirectional text	Bahut powerful, lekin bugs the
CSS Level 2.1	June 2011	CSS2 ke bugs fix kiye, browser compatibility improve ki, selectors better hue	Sabse zyada use hone wala stable version tha
CSS Level 3 (CSS3)	1999–ongoing (modules se)	Modules system shuru hua (ek bada document nahi), Flexbox, Grid, Transitions, Animations, Shadows, Gradients, Media Queries, Border-radius, Transforms	Abhi bhi active – har module alag progress karta hai

Version / Level	Release Date	Kya Naya Aaya? (Main Features)	Status / Notes
CSS Level 4+	2015 se aage	Individual modules jaise Selectors Level 4, Color Level 5, Nesting, Container Queries, Scroll Snap	Modern features – jaise :has() selector, subgrid

Important baat: CSS3 ke baad **koi "CSS4" official nahi** hota. Log kabhi-kabhi mazak mein bol dete hain, lekin asal mein har cheez module ke level se jaati hai (jaise CSS Grid Level 2, CSS Color Module Level 5). Aaj (2026 mein) hum **CSS Snapshot** dekhte hain jo saare stable + ready modules ko collect karta hai.

Kuch Famous Modules Jo Aaj Sab Use Karte Hain

- **Selectors Level 3/4** → :nth-child, :not, :has()
- **Flexbox** → Flexible layouts ke liye best
- **CSS Grid** → 2D layouts banane ke liye powerful
- **Animations & Transitions** → Smooth effects
- **Media Queries** → Responsive design (mobile, tablet)
- **Variables** (--main-color: blue;)
- **Nesting** (2023–2025 mein native support aaya)

✅ CSS Syntax (CSS ka basic likhne ka tarika)

CSS ka har rule 3 main cheezon se banta hai:

CSS

```
selector {
  property: value;
}
```

- **Selector** → kis cheez ko style karna hai? (jaise button, heading, pura page etc.)
- **Property** → kya change karna hai? (jaise color, size, margin, font etc.)
- **Value** → us property ka naya maan kya hoga? (jaise red, 20px, bold etc.)

Important points:

- Property aur value ko **:** (colon) se separate karte hain
- Har line ke end mein **;** (semicolon) lagana zaroori hai (last property mein bhi lagao – safe practice)
- Pura rule **{ }** curly braces ke andar likhte hain

✅ CSS Comments (CSS mein comment kaise likhte hain?)

Comment likhne se code samajhne mein madad milti hai. Browser comment ko ignore kar deta hai.

CSS mein comment aise likhte hain:

CSS

```
/* Ye comment hai – browser ise nahi dekhta */
```

```
/*
  Ye multi-line comment hai
  Isme jitna chahe likh sakte ho
*/
```

Real Examples (2-3 achhe examples ke saath)

Example 1: Basic – Sab paragraphs ka text red karna

CSS

```
p {
   color: red;        /* property = color, value = red */
   font-size: 18px;     /* property = font-size, value = 18px */
}
```

Yahan **p** selector hai → matlab pura page ke sabhi <p> tags ko style milega.

Example 2: Ek specific button ko sundar banana

CSS

```
/* Ye comment hai – special button ke liye style */
.special-btn {
   background-color: #4CAF50;   /* green background */
   color: white;            /* white text */
   padding: 12px 24px;        /* upar-neeche 12px, left-right 24px space */
   border: none;            /* border hata diya */
   border-radius: 8px;        /* gol corners */
   font-size: 16px;          /* text size */
   cursor: pointer;          /* mouse aane pe haath ka sign */
}

.special-btn:hover {
   background-color: #45a049;   /* hover karne pe thoda dark green */
}
```

Example 3: Pura page ka background + heading style

CSS

```
/* Comment: body ka background change kar rahe hain */
body {
   background-color: #f0f2f5;   /* light grey background */
   font-family: Arial, sans-serif;
   margin: 0;               /* side mein koi extra space nahi */
}

/* Comment: sabhi headings (h1) ko style */
h1 {
   color: navy;
   text-align: center;
   padding: 20px;
   background-color: #e0f7fa;
```

```
}
```

Quick Summary Table (Ek nazar mein samajh aaye)

Cheez	Kya hota hai?	Example likhne ka tarika
Selector	Kis element ko style karna hai	p, .class, #id, h1
Property	Kya change karna hai	color, font-size, margin
Value	Naya maan kya hoga	blue, 16px, 20px 10px
Comment	Code samajhne ke liye note	/* yeh comment hai */

✅ CSS TEXT PROPERTIES –

---

1 color

👉 Text ka **rang (color)** set karta hai.

Values & Definition:

- **Color Name** → Predefined naam jaise red
- **HEX** → Hexadecimal color code
- **RGB** → Red, Green, Blue mix
- **RGBA** → RGB + opacity

Examples:

```
p { color: red; }
h1 { color: #0066ff; }
span { color: rgb(0, 128, 0); }
div { color: rgba(0, 0, 0, 0.6); }
```

---

2 text-align

👉 Text ko **horizontal direction** me align karta hai.

Values:

- left → Left side se align
- right → Right side se align
- center → Beech me
- justify → Dono side equal spacing

Examples:

```
p { text-align: left; }
h2 { text-align: center; }
div { text-align: right; }
article { text-align: justify; }
```

---

3 text-decoration

👉 Text par **line decoration** lagata hai.

Values:

- none → Koi line nahi
- underline → Neeche line

- overline → Upar line
- line-through → Beech se cut

Examples:

```
a { text-decoration: none; }
p { text-decoration: underline; }
h3 { text-decoration: overline; }
span { text-decoration: line-through; }
```

---

4 text-transform

👉 Text ke **letter case** ko change karta hai.

Values:

- uppercase → Sab capital
- lowercase → Sab small
- capitalize → Har word ka first letter capital

Examples:

```
p { text-transform: uppercase; }
h1 { text-transform: lowercase; }
div { text-transform: capitalize; }
```

---

5 text-indent

👉 Paragraph ki **first line ko aage se space** deta hai.

Values:

- px → Fixed space
- % → Parent width ke hisaab se
- em → Font-size ke hisaab se

Examples:

```
p { text-indent: 30px; }
p { text-indent: 10%; }
p { text-indent: 2em; }
```

---

6 line-height

👉 Lines ke beech ka **vertical gap** control karta hai.

Values:

- normal → Default spacing
- number → Font-size ka multiple
- px → Fixed height
- % → Percentage based

Examples:

```
p { line-height: normal; }
p { line-height: 1.6; }
p { line-height: 30px; }
p { line-height: 150%; }
```

---

7 letter-spacing

👉 Letters ke beech ka **space** set karta hai.

Values:

- normal → Default
- px → Extra space
- em → Relative space

Examples:

```
h1 { letter-spacing: normal; }
h1 { letter-spacing: 3px; }
h1 { letter-spacing: 0.2em; }
```

---

8 word-spacing

👉 Words ke beech ka **gap** control karta hai.

Values:

- normal → Default gap
- px → Custom gap
- em → Relative gap

Examples:

```
p { word-spacing: normal; }
p { word-spacing: 10px; }
p { word-spacing: 0.5em; }
```

---

9 white-space

👉 Text ke **spaces, wrapping aur line break** ko control karta hai.

Values:

- normal → Default wrap
- nowrap → Line break nahi
- pre → HTML jaisa likha waisa hi
- pre-wrap → Wrap + spaces maintain
- pre-line → Extra space ignore

Examples:

```
p { white-space: normal; }
p { white-space: nowrap; }
p { white-space: pre; }
p { white-space: pre-wrap; }
```

---

10 text-shadow

👉 Text ke peeche **shadow effect** deta hai.

Value Format:

horizontal vertical blur color

Examples:

```
h1 { text-shadow: 2px 2px black; }
```

```
h1 { text-shadow: 3px 3px 5px gray; }
h1 { text-shadow: -2px 2px 4px red; }
```

1️⃣1️⃣ direction

👉 Text ki **writing direction** batata hai.

Values:

- ltr → Left to Right
- rtl → Right to Left

Examples:

```
p { direction: ltr; }
p { direction: rtl; }
```

1️⃣2️⃣ unicode-bidi

👉 direction ke saath text behavior control karta hai.

Values:

- normal
- embed
- bidi-override

Examples:

```
p { unicode-bidi: normal; }
p { unicode-bidi: bidi-override; }
```

1️⃣3️⃣ text-overflow

👉 Jab text box se bahar jata hai.

Values:

- clip → Text cut ho jata hai
- ellipsis → ... show hota hai

Examples:

```
p { text-overflow: clip; }
p { text-overflow: ellipsis; }
```

🔥 **Sabse Important Rule (Ye yaad rakhna!)**

text-overflow tabhi kaam karega jab:

```
overflow: hidden;
white-space: nowrap;
```

1️⃣4️⃣ vertical-align

👉 Inline text ya elements ka **vertical alignment**.

Values:

- baseline
- middle

UNTOLD EDUCATION
Untold skills for modern jobs
CSS
Complete Notes

- top
- bottom
- super
- sub

Examples:

```
span { vertical-align: middle; }
span { vertical-align: super; }
```

1️⃣5️⃣ writing-mode

👉 Text ko **vertical ya horizontal** likhne ke liye.

Values:

- horizontal-tb
- vertical-rl
- vertical-lr

Examples:

```
p { writing-mode: horizontal-tb; }
p { writing-mode: vertical-rl; }
```

🔥 EXAM & INTERVIEW TIP

👉 Agar short answer aaye:

- text-align → alignment
- text-transform → letter case
- line-height → line spacing
- letter-spacing → character gap

✅ CSS FONT PROPERTIES –

1️⃣ font-family

👉 Text ke liye **font style / type** define karta hai.

Values & Definition:

- **Font Name** → Specific font jaise Arial
- **Generic Family** → Backup fonts
  (serif, sans-serif, monospace, cursive, fantasy)

📌 Browser agar first font nahi paya, to next use karega.

Examples:

```
p { font-family: Arial; }
h1 { font-family: "Times New Roman"; }
div { font-family: Verdana, sans-serif; }
code { font-family: monospace; }
```

2️⃣ font-size

👉 Text ka **size (height)** control karta hai.

Values & Definition:

- px → Fixed size
- em → Parent font-size ke relative
- rem → Root (html) ke relative
- % → Parent ke percentage
- Keywords → small, large

Examples:

```
p { font-size: 16px; }
h1 { font-size: 2em; }
div { font-size: 1.5rem; }
span { font-size: 120%; }
```

---

3 font-weight

👉 Text ki **motai (thickness)** control karta hai.

Values & Definition:

- normal → Normal thickness (400)
- bold → Bold text (700)
- lighter → Parent se halka
- bolder → Parent se zyada bold
- 100–900 → Numeric weight

Examples:

```
p { font-weight: normal; }
h2 { font-weight: bold; }
div { font-weight: 300; }
span { font-weight: 800; }
```

---

4 font-style

👉 Text ka **style / shape** set karta hai.

Values:

- normal → Normal text
- italic → Italic style
- oblique → Tilted text

Examples:

```
p { font-style: normal; }
h1 { font-style: italic; }
span { font-style: oblique; }
```

---

5 font-variant

👉 Text ko **small capital letters** me convert karta hai.

Values:

- normal → Default
- small-caps → Capital jaisa chhota text

Examples:

```
p { font-variant: normal; }
h2 { font-variant: small-caps; }
```

---

6 font-stretch

👉 Font ko **narrow ya wide** karta hai.

Values:

- ultra-condensed → Bahut patla
- condensed
- normal
- expanded
- ultra-expanded → Bahut wide

Examples:

```
p { font-stretch: normal; }
h1 { font-stretch: expanded; }
div { font-stretch: condensed; }
```

---

7 font-size-adjust

👉 Font readability ko **x-height ke basis par adjust** karta hai.

Values:

- none → Default
- number → Aspect ratio

Examples:

```
p { font-size-adjust: none; }
p { font-size-adjust: 0.58; }
```

📌 Rarely used, advanced typography me.

---

8 font-kerning

👉 Characters ke beech **automatic spacing** control karta hai.

Values:

- auto → Browser decide kare
- normal → Kerning apply
- none → Kerning band

Examples:

```
p { font-kerning: auto; }
h1 { font-kerning: normal; }
span { font-kerning: none; }
```

---

9 font-feature-settings

👉 OpenType fonts ke **advanced features** enable karta hai.

Values:

- normal
- "liga" → Ligatures
- "smcp" → Small caps

Examples:

```
p { font-feature-settings: normal; }
h1 { font-feature-settings: "liga"; }
div { font-feature-settings: "smcp"; }
```

🔟 font (Shorthand Property)

👉 Multiple font properties **ek line me likhne** ke liye.

Syntax:

font: style variant weight size/line-height family;

Examples:

```
p {
  font: italic normal bold 16px Arial;
}
h1 {
  font: normal small-caps 700 2em "Times New Roman";
}
div {
  font: italic 18px/30px Verdana;
}
```

📌 font-size aur font-family **mandatory** hote hain.

🔥 EXAM & INTERVIEW MOST IMPORTANT

✅ font-family
✅ font-size
✅ font-weight
✅ font-style
✅ font-variant
✅ font (shorthand)

✅ CSS UNITS –

CSS Units ka use **size, spacing, position, font-size, width, height** define karne ke liye hota hai.

CSS Units **2 main types** ki hoti hain 👇

🔷 1. ABSOLUTE UNITS

👉 Ye **fix / constant** hoti hain.

Screen size ya parent element change ho, **value same rahti hai**.

🔹 px (Pixel)

Definition:

Screen ka sabse chhota visible dot.

📌 Most common & easy unit.

Examples:

```
p { font-size: 16px; }
div { width: 300px; }
```

🔹 cm (Centimeter)

Definition:

Real-world centimeter.

📌 Print media me use hota hai.

Examples:

```
div { width: 5cm; }
p { margin: 1cm; }
```

🔹 mm (Millimeter)

Definition:

1 cm = 10 mm

Examples:

```
div { padding: 10mm; }
p { border-width: 2mm; }
```

🔹 in (Inch)

Definition:

1 inch = 2.54 cm

Examples:

```
div { width: 2in; }
p { margin: 0.5in; }
```

🔹 pt (Point)

Definition:

1 pt = 1/72 inch

Mostly **printing** ke liye.

Examples:

```
p { font-size: 12pt; }
h1 { font-size: 24pt; }
```

🔹 pc (Pica)

Definition:

1 pc = 12 pt

Examples:

```
p { font-size: 1pc; }
h2 { margin: 2pc; }
```

🔷 2. RELATIVE UNITS

👉 Ye **relative** hoti hain — kisi **parent, root, viewport ya font-size** par depend karti hain.
📌 Responsive design ke liye best.

🔷 em

Definition:
Parent element ke **font-size** ke relative.
📌 1em = parent font-size
Examples:
div { font-size: 20px; }

```
p { font-size: 1.5em; }   /* 30px */
p { margin: 2em; }
```

⚠️ Nesting me value multiply hoti hai.

🔷 rem

Definition:
Root (html) element ke font-size ke relative.
📌 1rem = html font-size (mostly 16px)
Examples:

```
p { font-size: 1rem; }
h1 { font-size: 2.5rem; }
```

✅ em se zyada predictable.

🔷 % (Percentage)

Definition:
Parent element ke size ke percentage ke hisaab se.
Examples:

```
div { width: 50%; }
p { font-size: 120%; }
```

🔷 vw (Viewport Width)

Definition:
Viewport (screen) width ka 1%.
📌 100vw = full screen width
Examples:

```
div { width: 100vw; }
h1 { font-size: 5vw; }
```

🔷 vh (Viewport Height)

Definition:
Viewport height ka 1%.

📌 100vh = full screen height

Examples:

```
section { height: 100vh; }
div { min-height: 50vh; }
```

🔷 vmin

Definition:
Viewport ke **smaller side** ka 1%.

Examples:

```
div { font-size: 5vmin; }
section { padding: 10vmin; }
```

🔷 vmax

Definition:
Viewport ke **larger side** ka 1%.

Examples:

```
h1 { font-size: 5vmax; }
div { margin: 8vmax; }
```

🔷 ch

Definition:
Font ke 0 (zero) character ki width ke equal.

📌 Typewriter / input fields me use hota hai.

Examples:

```
input { width: 20ch; }
p { max-width: 60ch; }
```

🔷 ex

Definition:
Font ke lowercase x ki height.

📌 Rarely used.

Examples:

```
p { font-size: 2ex; }
span { vertical-align: 1ex; }
```

🔷 fr (Fraction – Grid only)

Definition:
Available space ka fraction (CSS Grid).

Examples:

```
grid-template-columns: 1fr 2fr;
grid-template-rows: 3fr 1fr;
```

📌 **EXAM-IMPORTANT SUMMARY**

Use Case	Best Unit
Font size	rem
Spacing	em, rem
Layout width	%, vw
Full screen section	vh
Grid layout	fr
Fixed size	px

✅ CSS BACKGROUND –

1️⃣ **background-color**

📌 **Property Definition**

Element ke **peeche ka background color** set karta hai.

🔷 **Values & Unki Definitions**

🔶 **Color Name (red, blue)**

Predefined color names jo browser samajhta hai.

```
div { background-color: red; }
section { background-color: blue; }
```

🔶 **HEX (#RRGGBB)**

Hexadecimal color code

RR = Red, GG = Green, BB = Blue

```
div { background-color: #ff0000; }
section { background-color: #00ffcc; }
```

🔶 **RGB (rgb(r,g,b))**

Red, Green, Blue values (0–255)

```
div { background-color: rgb(255, 0, 0); }
section { background-color: rgb(0, 200, 100); }
```

🔶 **RGBA (rgba(r,g,b,a))**

RGB + **alpha (opacity)**

a = 0 (transparent) → 1 (solid)

```
div { background-color: rgba(0, 0, 0, 0.5); }
section { background-color: rgba(255, 0, 0, 0.3); }
```

UNTOLD EDUCATION
Untold skills for modern jobs
CSS
Complete Notes
Styling modern websites from basics to advanced
Created by Pawan Soni

2 background-image

📌 **Property Definition**

Background me **image lagata hai**.

🔷 **Values & Definitions**

🔶 **url()**

Image ka path batata hai.

```
div { background-image: url("bg.jpg"); }
section { background-image: url("images/pattern.png"); }
```

🔶 **none**

Koi background image nahi.

```
div { background-image: none; }
p { background-image: none; }
```

3 background-repeat

📌 **Property Definition**

Background image **repeat hogi ya nahi** aur **kis direction me** hogi.

🔷 **Values & Definitions**

🔶 **repeat**

Image **horizontal + vertical dono me repeat** hoti hai (default).

```
div { background-repeat: repeat; }
section { background-repeat: repeat; }
```

🔶 **no-repeat**

Image **sirf ek baar** show hogi.

```
div { background-repeat: no-repeat; }
section { background-repeat: no-repeat; }
```

🔶 **repeat-x**

Image **sirf left → right** repeat hogi.

```
div { background-repeat: repeat-x; }
section { background-repeat: repeat-x; }
```

🔶 **repeat-y**

Image **sirf top → bottom** repeat hogi.

```
div { background-repeat: repeat-y; }
section { background-repeat: repeat-y; }
```

🔶 **space**

Images ke beech **equal space chhod ke repeat** hoti hain.

```
div { background-repeat: space; }
section { background-repeat: space; }
```

🔶 **round**

Image ko **stretch karke repeat** karta hai.

```
div { background-repeat: round; }
section { background-repeat: round; }
```

4️⃣ **background-position**

📌 **Property Definition**

Background image **element ke andar kis jagah lagegi**, ye batata hai.

🔷 **Values & Definitions**

🔶 **Keywords (left, right, top, bottom, center)**

Image ko fixed position dete hain.

```
div { background-position: center; }
section { background-position: top right; }
```

🔶 **Percentage (%)**

Element ke size ke according position.

```
div { background-position: 50% 50%; }
section { background-position: 20% 80%; }
```

🔶 **Length (px)**

Exact distance se position.

```
div { background-position: 20px 40px; }
section { background-position: 100px 10px; }
```

5️⃣ **background-size**

📌 **Property Definition**

Background image ka **size control** karta hai.

🔷 **Values & Definitions**

🔶 **auto**

Original image size.

```
div { background-size: auto; }
section { background-size: auto; }
```

🔶 **cover**

Image **poore element ko cover** karegi
(image crop ho sakti hai)

```
div { background-size: cover; }
section { background-size: cover; }
```

🔶 **contain**

Image **poori dikhai degi**, crop nahi hogi.

```
div { background-size: contain; }
section { background-size: contain; }
```

🔶 **Length (px)**

Fixed width & height.

```
div { background-size: 200px 100px; }
section { background-size: 300px 150px; }
```

🔶 **Percentage (%)**

Element ke size ke hisaab se.

```
div { background-size: 50% 50%; }
section { background-size: 100% 80%; }
```

6️⃣ **background-attachment**

📌 **Property Definition**

Background **scroll ke saath move kare ya fixed rahe**.

🔷 **Values & Definitions**

🔶 **scroll**

Background element ke saath scroll karta hai (default).

```
body { background-attachment: scroll; }
div { background-attachment: scroll; }
```

🔶 **fixed**

Background **screen ke saath fixed** rahta hai.

```
body { background-attachment: fixed; }
section { background-attachment: fixed; }
```

🔶 **local**

Background **element ke andar scroll** karta hai.

```
div { background-attachment: local; }
section { background-attachment: local; }
```

7️⃣ **background-origin**

📌 **Property Definition**

Background image **kahan se start hogi**.

🔷 **Values & Definitions**

🔶 **padding-box**

Padding ke andar se start hoti hai (default).

```
div { background-origin: padding-box; }
section { background-origin: padding-box; }
```

🔶 **border-box**

Border ke andar se start.

```
div { background-origin: border-box; }
section { background-origin: border-box; }
```

🔶 **content-box**

Sirf content area se start.

```
div { background-origin: content-box; }
section { background-origin: content-box; }
```

8️⃣ **background-clip**

📌 **Property Definition**

Background **kahan tak visible hogi**.

🔷 **Values & Definitions**

🔶 **border-box**

Border tak background dikhegi.

```
div { background-clip: border-box; }
section { background-clip: border-box; }
```

🔶 **padding-box**

Padding tak hi dikhegi.

```
div { background-clip: padding-box; }
section { background-clip: padding-box; }
```

🔶 **content-box**

Sirf content ke andar dikhegi.

```
div { background-clip: content-box; }
section { background-clip: content-box; }
```

9️⃣ **background-blend-mode**

📌 **Property Definition**

Multiple background layers **kaise mix hongi**.

🔷 **Values & Definitions**

- normal → Normal mix

- multiply → Dark mix
- screen → Light mix
- overlay → Contrast mix
- darken → Dark areas strong
- lighten → Light areas strong

div { background-blend-mode: multiply; }
section { background-blend-mode: overlay; }

🔟 background (Shorthand)

📌 **Property Definition**

Saari background properties **ek line me likhne** ke liye.

```
div {
  background: red url("bg.jpg") no-repeat center/cover;
}
body {
  background: linear-gradient(to right, red, blue);
}
```

✅ CSS BORDER –

1️⃣ border-style

📌 **Property Definition**

Border **kis type ki line** me dikhegi (solid, dotted, etc.) ye decide karta hai.

⚠️ Border tabhi dikhegi jab border-style diya ho.

🔷 **Values & Unki Definitions**

🔶 **none**

Koi border nahi (default).

```
div { border-style: none; }
p { border-style: none; }
```

🔶 **solid**

Seedhi continuous line.

```
div { border-style: solid; }
section { border-style: solid; }
```

🔶 **dotted**

Dots ki form me border.

```
div { border-style: dotted; }
```

p { border-style: dotted; }

🔶 **dashed**

Chhoti–chhoti lines me border.

```
div { border-style: dashed; }
section { border-style: dashed; }
```

🔶 **double**

Do parallel solid lines.

```
div { border-style: double; }
p { border-style: double; }
```

🔶 **groove**

3D groove (andar dhansi hui).

```
div { border-style: groove; }
section { border-style: groove; }
```

🔶 **ridge**

3D ridge (bahar nikli hui).

```
div { border-style: ridge; }
p { border-style: ridge; }
```

🔶 **inset**

Border element ke andar dabi hui lagti hai.

```
div { border-style: inset; }
section { border-style: inset; }
```

🔶 **outset**

Border element ke bahar uthi hui lagti hai.

```
div { border-style: outset; }
p { border-style: outset; }
```

2️⃣ **border-width**

📌 **Property Definition**

Border **kitni moti (thick)** hogi, ye batata hai.

🔷 **Values & Definitions**

🔶 **Length (px)**

Exact thickness.

```
div { border-width: 2px; }
section { border-width: 5px; }
```

🔶 **Keywords**

- thin → Patli border
- medium → Normal border

- thick → Moti border

```
div { border-width: thin; }
p { border-width: thick; }
```

🔶 **Side-wise values**

Top, Right, Bottom, Left alag-alag.

```
div { border-width: 2px 4px 6px 8px; }
section { border-width: 1px 3px; }
```

3️⃣ **border-color**

📌 **Property Definition**

Border ka **rang (color)** set karta hai.

🔷 **Values & Definitions**

🔶 **Color Name**

Predefined color.

```
div { border-color: red; }
section { border-color: blue; }
```

🔶 **HEX**

Hexadecimal color code.

```
div { border-color: #ff0000; }
p { border-color: #00ff99; }
```

🔶 **RGB**

Red, Green, Blue values.

```
div { border-color: rgb(255, 0, 0); }
section { border-color: rgb(0, 0, 255); }
```

🔶 **RGBA**

RGB + opacity.

```
div { border-color: rgba(0, 0, 0, 0.5); }
p { border-color: rgba(255, 0, 0, 0.3); }
```

4️⃣ **border (Shorthand Property)**

📌 **Property Definition**

border-width + border-style + border-color

👉 Sab ek hi line me likhne ke liye.

Examples:

```
div { border: 2px solid red; }
section { border: thick dashed blue; }
```

```
p { border: 4px double #333; }
```

5 border-radius

📌 Property Definition

Border ke **corners ko gol (round)** karta hai.

🔷 Values & Definitions

🔶 Length (px)

Fixed roundness.

```
div { border-radius: 10px; }
section { border-radius: 25px; }
```

🔶 Percentage (%)

Element ke size ke according round.

```
div { border-radius: 50%; }   /* circle */
section { border-radius: 20%; }
```

🔶 Side-wise radius

Har corner alag.

```
div { border-radius: 10px 20px 30px 40px; }
p { border-radius: 15px 5px; }
```

---

6 Individual Side Borders

📌 Property Definition

Har side ki border alag-alag control karne ke liye.

🔷 border-top

```
div { border-top: 2px solid red; }
section { border-top: 4px dashed blue; }
```

🔷 border-right

```
div { border-right: 3px dotted green; }
p { border-right: 5px solid black; }
```

🔷 border-bottom

```
div { border-bottom: 2px double purple; }
section { border-bottom: 6px solid orange; }
```

🔷 border-left

div { border-left: 4px solid brown; }
p { border-left: 1px dashed gray; }

7️⃣ Individual Border Properties (Side-wise)

📌 **Property Definition**

Har side ka **style, width, color** alag set kar sakte hain.

Examples:

```
div { border-top-style: solid; }
div { border-right-style: dashed; }
section { border-bottom-width: 5px; }
section { border-left-width: thick; }
p { border-top-color: red; }
p { border-bottom-color: blue; }
```

8️⃣ border-image

📌 **Property Definition**

Image ko border ki jagah use karta hai.

🔹 border-image-source

Border image ka path.

```
div { border-image-source: url("border.png"); }
section { border-image-source: url("frame.png"); }
```

🔹 border-image-slice

Image ko kitne parts me cut kare.

```
div { border-image-slice: 30; }
section { border-image-slice: 20%; }
```

🔹 border-image-width

Image border ki width.

```
div { border-image-width: 10px; }
p { border-image-width: 2em; }
```

🔹 border-image-repeat

Image repeat kaise hogi.

Values:

- stretch → stretch kare
- repeat → repeat kare
- round → fit karke repeat
- space → space ke saath repeat

```
div { border-image-repeat: round; }
section { border-image-repeat: stretch; }
```

🔹 border-image (Shorthand)

```
div {
  border: 10px solid transparent;
  border-image: url("border.png") 30 round;
}
```

✅ CSS MARGIN –

📌 **MARGIN KYA HOTA HAI?**

👉 **Margin element ke bahar ka space** hota hai.

👉 Ye element aur dusre elements ke beech **distance** banata hai.

👉 Margin **transparent hota hai** (background color nahi hota).

1️⃣ **margin (Shorthand Property)**

📌 **Property Definition**

Element ke **charo taraf (top, right, bottom, left)** ka margin **ek saath** set karta hai.

🔷 **Values & Definitions**

🔶 **Single Value**

Sabhi sides par same margin.

```
div { margin: 20px; }
section { margin: 5em; }
```

🔶 **Two Values**

```
top-bottom | left-right
div { margin: 10px 30px; }
section { margin: 5% 15%; }
```

🔶 **Three Values**

```
top | left-right | bottom
div { margin: 10px 20px 30px; }
section { margin: 1em 2em 3em; }
```

🔶 **Four Values**

```
top | right | bottom | left
div { margin: 10px 20px 30px 40px; }
section { margin: 5px 15px 25px 35px; }
```

2️⃣ **margin-top**

📌 **Property Definition**

Element ke **upar ka external space** set karta hai.

🔷 **Values & Definitions**

🔶 **Length (px, em, rem)**

Fixed ya relative space.

```
div { margin-top: 20px; }
section { margin-top: 2em; }
```

🔶 **Percentage (%)**

Parent element ke **width** ke according.

```
div { margin-top: 10%; }
section { margin-top: 5%; }
```

🔶 **auto**

Browser khud calculate karta hai (rare for top).

```
div { margin-top: auto; }
section { margin-top: auto; }
```

🔶 **Negative Value**

Element ko **upar ki taraf khinchta** hai.

```
div { margin-top: -20px; }
section { margin-top: -2em; }
```

3️⃣ **margin-right**

📌 **Property Definition**

Element ke **right side ka bahari space**.

🔷 **Values & Definitions**

```
div { margin-right: 30px; }
section { margin-right: 3em; }
div { margin-right: 10%; }
section { margin-right: auto; }
div { margin-right: -15px; }
section { margin-right: -1em; }
```

4️⃣ **margin-bottom**

📌 **Property Definition**

Element ke **neeche ka external space**.

🔷 **Values & Definitions**

```
div { margin-bottom: 25px; }
section { margin-bottom: 4em; }
div { margin-bottom: 15%; }
section { margin-bottom: auto; }
```

```
div { margin-bottom: -10px; }
section { margin-bottom: -2em; }
```

---

5 margin-left

📌 **Property Definition**

Element ke **left side ka bahari space**.

🔷 **Values & Definitions**

```
div { margin-left: 40px; }
section { margin-left: 5em; }
div { margin-left: 20%; }
section { margin-left: auto; }
div { margin-left: -30px; }
section { margin-left: -1.5em; }
```

---

6 auto (Margin ka special value)

📌 **Value Definition**

Browser **khud space calculate** karta hai.

📌 Mostly **horizontal centering** ke liye use hota hai.

Examples:

```
div {
  width: 300px;
  margin: auto;
}
section {
  width: 50%;
  margin-left: auto;
  margin-right: auto;
}
```

---

7 Negative Margin (IMPORTANT)

📌 **Definition**

Negative margin element ko **dusre element ke upar laata** hai.

Examples:

```
div { margin-top: -50px; }
section { margin-left: -100px; }
```

⚠️ Advanced layouts me use hota hai.

---

8 Margin Collapsing (Concept – Exam Favourite)

📌 **Definition**

Vertical margins (top & bottom) **merge ho jaate hain**, add nahi hote.

```
p { margin-bottom: 30px; }
div { margin-top: 20px; }
```

➡️ Actual gap = **30px**, 50px nahi.

📌 Sirf **vertical margins** me hota hai.

❌ **COMMON CONFUSION (Clear rakho)**

❌ Margin = element ke **bahar**

❌ Padding = element ke **andar**

❌ Margin me **background color nahi hota**

📊 **QUICK EXAM TABLE**

Property	Kaam
margin	All sides
margin-top	Upar
margin-right	Right
margin-bottom	Neeche
margin-left	Left

✅ CSS PADDING –

📌 **PADDING KYA HOTA HAI?**

👉 **Padding element ke andar ka space** hota hai.

👉 Ye **content aur border ke beech** distance banata hai.

👉 Padding me **background color visible hota hai** (margin se ye main difference hai).

1️⃣ **padding (Shorthand Property)**

📌 **Property Definition**

Element ke **charo sides (top, right, bottom, left)** ka padding **ek saath** set karta hai.

🔷 **Values & Unki Definitions**

🔶 **Single Value**

👉 Sabhi sides par **same padding** lagti hai.

```
div { padding: 20px; }
section { padding: 2em; }
```

🔶 **Two Values**

👉 top-bottom | left-right

```
div { padding: 10px 30px; }
```

```
section { padding: 5% 15%; }
```

◆ **Three Values**

👉 top | left-right | bottom

```
div { padding: 10px 20px 30px; }
section { padding: 1em 2em 3em; }
```

◆ **Four Values**

👉 top | right | bottom | left

```
div { padding: 10px 20px 30px 40px; }
section { padding: 5px 15px 25px 35px; }
```

---

2 padding-top

📌 **Property Definition**

Element ke **content ke upar ka internal space** set karta hai.

🔷 **Values & Definitions**

◆ **Length (px, em, rem)**

👉 Fixed ya relative padding.

```
div { padding-top: 20px; }
section { padding-top: 2em; }
```

◆ **Percentage (%)**

👉 Parent element ke **width** ke according padding.

```
div { padding-top: 10%; }
section { padding-top: 5%; }
```

◆ **Zero (0)**

👉 Koi padding nahi.

```
div { padding-top: 0; }
section { padding-top: 0px; }
```

---

3 padding-right

📌 **Property Definition**

Element ke **content ke right side ka internal space**.

🔷 **Values & Definitions**

```
div { padding-right: 30px; }
section { padding-right: 3em; }
div { padding-right: 15%; }
section { padding-right: 5%; }
div { padding-right: 0; }
```

```
section { padding-right: 0px; }
```

---

4 padding-bottom

📌 **Property Definition**

Element ke **content ke neeche ka internal space**.

🔷 **Values & Definitions**

```
div { padding-bottom: 25px; }
section { padding-bottom: 4em; }
div { padding-bottom: 10%; }
section { padding-bottom: 6%; }
div { padding-bottom: 0; }
section { padding-bottom: 0px; }
```

---

5 padding-left

📌 **Property Definition**

Element ke **content ke left side ka internal space**.

🔷 **Values & Definitions**

```
div { padding-left: 40px; }
section { padding-left: 5em; }
div { padding-left: 20%; }
section { padding-left: 10%; }
div { padding-left: 0; }
section { padding-left: 0px; }
```

---

6 Allowed Values (Padding me kaun-kaun si value valid hai)

📌 **Valid Values & Definitions**

🔶 **Length Units**

px, em, rem, cm, mm

👉 Fixed ya relative space ke liye.

```
div { padding: 15px; }
section { padding: 1.5rem; }
```

🔶 **Percentage (%)**

👉 Parent element ki **width** ke base par.

```
div { padding: 8%; }
section { padding: 12%; }
```

🔶 **0**

👉 Padding completely remove kar deta hai.

div { padding: 0; }

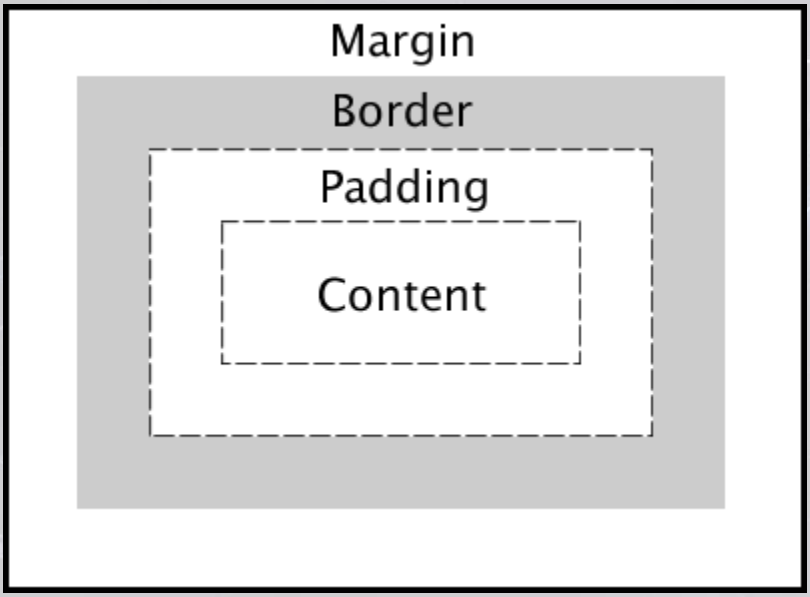
section { padding: 0px; }

⚠️ **IMPORTANT:**
❌ Padding me auto allowed nahi hota
❌ Padding negative nahi hoti

7️⃣ **Padding vs Margin (Exam Favourite)**

Point	Padding	Margin
Space	Inside element	Outside element
Background	Visible	Not visible
Negative	❌ Not allowed	✅ Allowed
auto	❌ Not allowed	✅ Allowed

✅ CSS BOX MODEL –



📦 **CSS BOX MODEL KYA HOTA HAI?**

CSS Box Model ke hisaab se **har HTML element ek box hota hai**, jisme 4 layers hoti hain:

1️⃣ **Content** – actual text / image
2️⃣ **Padding** – content ke andar ka space
3️⃣ **Border** – padding ke bahar line
4️⃣ **Margin** – element ke bahar ka space

```
Margin
└ Border
  └ Padding
    └ Content
```

🔷 BOX MODEL KI ALL PROPERTIES

1 width

📌 **Property Definition**

Element ke **content area ki width** set karta hai.

🔷 **Values & Definitions**

🔶 **Length (px, em, rem)**

Fixed ya relative width.

```
div { width: 200px; }
section { width: 20em; }
```

🔶 **Percentage (%)**

Parent element ki width ke according.

```
div { width: 50%; }
section { width: 80%; }
```

🔶 **auto**

Browser khud width calculate karta hai (default).

```
div { width: auto; }
section { width: auto; }
```

2 height

📌 **Property Definition**

Element ke **content area ki height** set karta hai.

🔷 **Values & Definitions**

```
div { height: 150px; }
section { height: 10em; }
div { height: 60%; }
section { height: auto; }
```

3 padding

📌 **Property Definition**

Content aur border ke beech ka **inner space**.

🔷 Values & Definitions

```
div { padding: 20px; }
section { padding: 2em; }
div { padding: 10px 20px; }
section { padding: 5px 15px 25px 35px; }
```

📌 Allowed values: px, em, rem, %, 0

❌ auto & negative allowed nahi

4️⃣ border

📌 Property Definition

Padding ke bahar **line** jo element ko define karti hai.

🔷 Sub-properties

- border-width
- border-style
- border-color

Examples:

```
div { border: 2px solid red; }
section { border: 5px dashed blue; }
p {
  border-width: 4px;
  border-style: double;
  border-color: green;
}
```

5️⃣ margin

📌 Property Definition

Element ke bahar ka **external space**.

🔷 Values & Definitions

```
div { margin: 20px; }
section { margin: 3em; }
div { margin: 10px auto; }
section { margin: -20px; }
```

📌 Allowed: px, em, %, auto, negative values

📌 Background visible ❌ (transparent)

6️⃣ box-sizing ⭐ (MOST IMPORTANT)

📌 **Property Definition**

Width/height calculation **kaise hogi**, ye decide karta hai.

🔷 **Values & Definitions**

🔶 **content-box (Default)**

Width/height **sirf content** ki hoti hai.

Padding + border **add ho jaate hain**.

```
div {
  width: 200px;
  padding: 20px;
  box-sizing: content-box;
}
```

📌 Actual width = **240px**

🔶 **border-box**

Width/height me **padding + border included** hoti hai.

```
div {
  width: 200px;
  padding: 20px;
  box-sizing: border-box;
}
```

📌 Actual width = **200px** ✅ (BEST PRACTICE)

7️⃣ min-width

📌 **Property Definition**

Element ki **minimum width** fix karta hai.

Examples:

```
div { min-width: 300px; }
section { min-width: 40%; }
```

8️⃣ max-width

📌 **Property Definition**

Element ki **maximum width limit** set karta hai.

Examples:

```
div { max-width: 600px; }
```

```
section { max-width: 90%; }
```

9️⃣ min-height

📌 **Property Definition**

Element ki **minimum height**.

Examples:

```
div { min-height: 200px; }
section { min-height: 50vh; }
```

🔟 max-height

📌 **Property Definition**

Element ki **maximum height limit**.

Examples:

```
div { max-height: 400px; }
section { max-height: 80vh; }
```

1️⃣1️⃣ overflow (Box Model se directly related)

📌 **Property Definition**

Content box se bahar nikle to kya hoga.

🔷 **Values & Definitions**

🔶 **visible**

Content bahar dikhega (default).

```
div { overflow: visible; }
section { overflow: visible; }
```

🔶 **hidden**

Extra content chhup jaata hai.

```
div { overflow: hidden; }
section { overflow: hidden; }
```

🔶 **scroll**

Scroll bar hamesha dikhegi.

```
div { overflow: scroll; }
section { overflow: scroll; }
```

UNTOLD EDUCATION

🔶 **auto**

Zarurat ho to scroll bar.

```
div { overflow: auto; }
section { overflow: auto; }
```

📊 **BOX MODEL TOTAL SIZE FORMULA (EXAM GOLD)**

🔷 **Default (content-box)**

Total Width =

width + padding-left + padding-right + border-left + border-right

---

🔷 **border-box**

Total Width = width (padding + border included)

✅ CSS SELECTORS –

📌 **CSS SELECTOR KYA HOTA HAI?**

👉 CSS selector batata hai ki **HTML ke kaunse element par CSS apply hogi**.

```
selector {
  property: value;
}
```

🔷 1️⃣ **Universal Selector ***

📌 **Definition**

Page ke **saare elements** ko select karta hai.

🔷 **Format (Value)**

- * → all elements

Examples:

```
* { margin: 0; }
* { box-sizing: border-box; }
```

🔷 2️⃣ **Element (Tag) Selector**

📌 **Definition**

HTML ke **specific tag** ko select karta hai.

🔷 **Value**

- Tag name (p, div, h1)

Examples:

```
p { color: red; }
h1 { font-size: 32px; }
```

🔷 3️⃣ Class Selector .class

📌 **Definition**

Same class wale **multiple elements** ko select karta hai.

🔷 **Value**

- .classname

Examples:

```
.box { background: yellow; }
.card { padding: 20px; }
```

🔷 4️⃣ ID Selector #id

📌 **Definition**

Ek **unique element** ko select karta hai.

🔷 **Value**

- #idname

Examples:

```
#header { height: 100px; }
#login { border: 2px solid red; }
```

⚠️ Ek page me ID ek hi baar use honi chahiye.

🔷 5️⃣ Group Selector

📌 **Definition**

Multiple selectors ko **ek saath** select karta hai.

🔷 **Value**

- selector1, selector2

Examples:

```
h1, h2, h3 { color: blue; }
p, div, span { font-family: Arial; }
```

🔷 6️⃣ Descendant Selector (Space)

📌 **Definition**

Parent ke **andar ke sab child elements** select karta hai.

🔷 **Value**

- parent child

Examples:

```
div p { color: red; }
section span { font-weight: bold; }
```

🔷 7️⃣ Child Selector >

📌 **Definition**

Sirf **direct child** ko select karta hai.

🔷 **Value**

- parent > child

Examples:

```
div > p { color: blue; }
ul > li { list-style: none; }
```

🔷 8️⃣ Adjacent Sibling Selector +

📌 **Definition**

Sirf **turant next sibling** ko select karta hai.

🔷 **Value**

- element + element

Examples:

```
h1 + p { color: green; }
div + section { margin-top: 20px; }
```

🔷 9️⃣ General Sibling Selector ~

📌 **Definition**

Same parent ke **saare next siblings** select karta hai.

🔷 **Value**

- element ~ element

Examples:

```
h1 ~ p { color: orange; }
label ~ input { border: 1px solid red; }
```

🔷 🔟 Attribute Selector

📌 **Definition**

HTML attribute ke base par element select karta hai.

🔷 **Values & Definitions**

🔶 **[attr]**

Jisme attribute ho.

```
input[type] { border: 2px solid black; }
a[target] { color: red; }
```

◆ [attr="value"]

Exact value match.

```
input[type="text"] { background: yellow; }
a[target="_blank"] { color: blue
```

◆ [attr~="value"]

Word match (space separated).

```
div[class~="box"] { border: 1px solid red; }
p[class~="highlight"] { background: pink; }
```

◆ [attr^="value"]

Value **start hoti ho** isse.

```
a[href^="https"] { color: green; }
img[src^="img"] { border: 2px solid blue; }
```

◆ [attr$="value"]

Value **end hoti ho** isse.

```
a[href$=".pdf"] { color: red; }
img[src$=".jpg"] { border-radius: 10px; }
```

◆ [attr*="value"]

Value ke **beech me ho**.

```
a[href*="google"] { color: purple; }
div[class*="card"] { box-shadow: 0 0 10px gray; }
```

◆ 1 1 Pseudo-Class Selector :

📌 Definition

Element ki **state (condition)** ke base par select karta hai.

◆ Common Values & Definitions

◆ :hover

Mouse le jaane par.

```
a:hover { color: red; }
button:hover { background: black; }
```

UNTOLD EDUCATION
Untold skills for modern jobs
CSS
Complete Notes

◆ :active

Click karte time.

```
button:active { background: blue; }
a:active { color: green; }
```

◆ :focus

Input me cursor aane par.

```
input:focus { border-color: red; }
textarea:focus { background: yellow; }
```

◆ :first-child

Parent ka pehla child.

```
li:first-child { color: red; }
p:first-child { font-weight: bold; }
```

◆ :nth-child(n)

Nth position ka element.

```
li:nth-child(2) { color: blue; }
tr:nth-child(even) { background: #eee; }
```

◆ 1 2 Pseudo-Element Selector ::

📌 Definition

Element ke **specific part** ko style karta hai.

◆ Values & Definitions

◆ ::before

Content se pehle add karta hai.

```
p::before { content: "👉 "; }
h1::before { content: "★ "; }
```

◆ ::after

Content ke baad add karta hai.

```
p::after { content: " ✓"; }
h2::after { content: " 🔥"; }
```

◆ ::first-letter

Pehla letter.

```
p::first-letter { font-size: 30px; }
article::first-letter { color: red; }
```

◆ ::first-line

Pehli line.

```
p::first-line { font-weight: bold; }
div::first-line { color: blue; }
```

◆ 1 3 Combination Selectors

📌 Definition

Multiple selectors ko **combine** karke use karna.

Examples:

```
div.box p { color: red; }
ul.menu > li.active { background: yellow; }
```

✅ CSS POSITION –

📌 CSS POSITION KYA HOTA HAI?

👉 position property batati hai ki **element page par kaise place hoga**

👉 Aur top / right / bottom / left se **uski exact jagah** decide hoti hai

---

◆ 1 position

📌 Property Definition

Element ki **positioning type** define karta hai.

◆ Values & Unki Definitions

◆ static (DEFAULT)

Definition:

Element **normal document flow** me hota hai.

top, right, bottom, left, z-index ka **koi effect nahi hota**.

```
div {
  position: static;
  top: 50px;
}
p {
  position: static;
```

```
}
```

🔷 relative

Definition:

Element **apni original position se relative** move karta hai.

Original space **reserve rehta hai**.

```
div {
  position: relative;
  top: 20px;
}
section {
  position: relative;
  left: 30px;
}
```

📌 Mostly **parent banane** ke liye use hota hai (absolute ke saath).

🔷 absolute

Definition:

Element **normal flow se bahar** ho jata hai.

Nearest **positioned parent** ke according place hota hai.

Agar parent nahi mile → body ke according.

```
div {
  position: absolute;
  top: 10px;
  right: 20px;
}
.box {
  position: absolute;
  bottom: 0;
  left: 0;
}
```

⚠️ Space reserve **nahi hota**.

🔷 fixed

Definition:

Element **viewport (screen)** ke according fixed hota hai.

Scroll karne par bhi **move nahi hota**.

```
header {
  position: fixed;
  top: 0;
```

```
}
button {
  position: fixed;
  bottom: 20px;
  right: 20px;
}
```

📌 Sticky header / chat button ke liye use hota hai.

🔷 **sticky**

Definition:

Element pehle **relative** ki tarah behave karta hai,
scroll ke baad **fixed** ho jata hai.

```
nav {
  position: sticky;
  top: 0;
}
h2 {
  position: sticky;
  top: 50px;
}
```

⚠️ top / bottom / left / right dena **mandatory** hai.

🔷 2️⃣ **top**

📌 **Property Definition**

Element ko **upar se kitna distance** chahiye.

Values & Examples

```
div {
  position: relative;
  top: 20px;
}
.box {
  position: absolute;
  top: 10%;
}
header {
  position: fixed;
  top: 0;
}
```

🔷 3 right

📌 **Property Definition**

Element ko **right side se kitna distance**.

```
div {
 position: relative;
 right: 30px;
}
.box {
 position: absolute;
 right: 10%;
}
```

🔷 4 bottom

📌 **Property Definition**

Element ko **neeche se kitna distance**.

```
div {
 position: absolute;
 bottom: 0;
}
button {
 position: fixed;
 bottom: 20px;
}
```

🔷 5 left

📌 **Property Definition**

Element ko **left side se kitna distance**.

```
div {
 position: relative;
 left: 40px;
}
.box {
 position: absolute;
 left: 5%;
}
```

🔷 6 z-index

📌 **Property Definition**

Elements ke **upar-neeche (layer order)** ko control karta hai.

👉 Jis ka z-index zyada, wo upar dikhega.

⚠️ Sirf **positioned elements** par kaam karta hai
(relative / absolute / fixed / sticky)

Values & Examples

```
div {
  position: relative;
  z-index: 1;
}
.box {
  position: absolute;
  z-index: 10;
}
.modal {
  position: fixed;
  z-index: 999;
}
```

🔷 7️⃣ **inset (Shortcut – Modern CSS)**

📌 **Property Definition**

top + right + bottom + left ko **ek hi line** me likhne ke liye.

Examples

```
div {
  position: absolute;
  inset: 10px;
}
.box {
  position: fixed;
  inset: 20px 40px;
}
.card {
  position: absolute;
  inset: 10px 20px 30px 40px;
}
```

📊 **POSITION SUMMARY TABLE (EXAM GOLD)**

Position	Flow me?	Reference
static	✅ yes	normal flow
relative	✅ yes	itself
absolute	❌ no	parent/body
fixed	❌ no	viewport
sticky	✅ → ❌	scroll based

✅ CSS z-index –

📌 **z-index KYA HOTA HAI?**

👉 z-index **elements ke upar–neeche (layer order)** ko control karta hai.

👉 Socho jaise **layers / floors**:

- **z-index zyada** → element **upar**
- **z-index kam** → element **neeche**

📌 Ye **sirf tab kaam karta hai jab element positioned ho**.

🔷 **BASIC RULE (EXAM GOLD)**

z-index kaam karega sirf un elements par
jinme position ho:
relative | absolute | fixed | sticky

❌ position: static → z-index ignore ho jaata hai

1️⃣ **z-index PROPERTY**

📌 **Property Definition**

Element ka **stacking order** set karta hai (X–Y nahi, Z direction).

🔷 **Values & Unki Definitions**

🔶 **Positive Integer (1, 2, 10, 999)**

Definition:

Badi value = upar layer.

```
.box1 {
  position: relative;
  z-index: 1;
}
.box2 {
  position: relative;
  z-index: 10;
}
```

➡️ .box2 upar dikhega

◆ 0

Definition:

Default stacking level (positioned elements ke liye).

```
div {
  position: relative;
  z-index: 0;
}
.card {
  position: absolute;
  z-index: 0;
}
```

◆ Negative Value (-1, -10)

Definition:

Element **peeche chala jaata hai** (parent ke peeche bhi).

```
.bg {
  position: absolute;
  z-index: -1;
}
.shadow {
  position: relative;
  z-index: -5;
}
```

⚠️ Overuse mat karo, debugging mushkil hoti hai.

◆ auto (Default)

Definition:

Browser **khud decide** karta hai stacking order.

```
div {
  position: relative;
  z-index: auto;
```

```
}
section {
  position: absolute;
  z-index: auto;
}
```

2️⃣ Z-INDEX KAAM KAB KARTA HAI? (VERY IMPORTANT)

✅ **Ye sab par kaam karta hai:**

position: relative;

position: absolute;

position: fixed;

position: sticky;

❌ **Ye par kaam nahi karta:**

position: static; /* default */

❌ Example (NO EFFECT):

```
div {
  position: static;
  z-index: 999;
}
```

3️⃣ PRACTICAL LAYER EXAMPLE

```
.red {
  position: absolute;
  z-index: 1;
}

.blue {
  position: absolute;
  z-index: 2;
}
```

➡️ **Blue box red ke upar** dikhega

4️⃣ STACKING CONTEXT (MOST CONFUSING PART)

📌 **Stacking Context KYA HOTA HAI?**

👉 Jab koi element **naya "layer group" bana leta hai**, to uske andar ke z-index **bahar ke elements se compare nahi hote**.

🔷 **Naya stacking context kab banta hai?**

- position + z-index (auto ke alawa)

UNTOLD EDUCATION
Untold skills for modern jobs

- opacity < 1
- transform
- filter
- perspective
- will-change

🔥 **CONFUSION EXAMPLE**

```
.parent {
  position: relative;
  z-index: 1;
}
```

```
.child {
  position: absolute;
  z-index: 999;
}
.outside {
  position: relative;
  z-index: 10;
}
```

➡️ **.child 999 hone ke baad bhi .outside ke upar nahi jaa sakta**

kyonki .parent ne **alag stacking context** bana diya.

📌 Exam line:

Child ka z-index sirf **apne parent ke context** ke andar kaam karta hai.

5️⃣ **REAL LIFE USE CASES**

🔷 **Navbar upar rakhna**

```
nav {
  position: fixed;
  top: 0;
  z-index: 1000;
}
```

🔷 **Modal / Popup**

```
.modal {
  position: fixed;
  z-index: 9999;
}
```

🔷 **Background image peeche**

```
.bg-image {
  position: absolute;
  z-index: -1;
}
```

📌 CSS GRADIENT

👉 Gradient ka matlab:

ek color se doosre color me smooth transition

👉 Gradient:

- ❌ image file nahi hota
- ✅ background-image ka part hota hai
- ✅ lightweight + responsive hota hai

🔷 CSS GRADIENT KE MAIN TYPES

1️⃣ linear-gradient()

2️⃣ radial-gradient()

3️⃣ conic-gradient()

🔁 Inke **repeating versions** bhi hote hain:

- repeating-linear-gradient()
- repeating-radial-gradient()
- repeating-conic-gradient()

1️⃣ LINEAR GRADIENT

📌 **Definition**

Colors **seedhi line (straight line)** me change hote hain.

📌 **Syntax**

background-image: linear-gradient(direction/angle, color-stop1, color-stop2, ...);

🔷 **Direction VALUES (with definition)**

🔶 **Default (top → bottom)**

Agar direction na likho to upar se neeche.

```
div {
  background-image: linear-gradient(red, yellow);
}
section {
  background-image: linear-gradient(blue, white);
```

```
}
```

◆ to right

Left → Right color flow

```
div {
  background-image: linear-gradient(to right, red, blue);
}
section {
  background-image: linear-gradient(to right, black, gray);
}
```

◆ to left

Right → Left color flow

```
div {
  background-image: linear-gradient(to left, green, yellow);
}
section {
  background-image: linear-gradient(to left, orange, red);
}
```

◆ to top

Bottom → Top

```
div {
  background-image: linear-gradient(to top, purple, pink);
}
section {
  background-image: linear-gradient(to top, navy, skyblue);
}
```

◆ Diagonal Directions

```
div {
  background-image: linear-gradient(to bottom right, red, blue);
}
section {
  background-image: linear-gradient(to top left, black, white);
}
```

◆ Angle VALUES (deg) – Definition

👉 Angle se direction define hoti hai

Angle	Direction
0deg	bottom → top
90deg	left → right
180deg	top → bottom
270deg	right → left

```
div {
  background-image: linear-gradient(45deg, red, yellow);
}
section {
  background-image: linear-gradient(135deg, blue, purple);
}
```

🔷 **Color Stops (%) – Definition**

👉 Har color **kitni jagah lega**

```
div {
  background-image: linear-gradient(
    to right,
    red 0%,
    yellow 50%,
    green 100%
  );
}
section {
  background-image: linear-gradient(
    blue 20%,
    white 80%
  );
}
```

🔁 1️⃣ **REPEATING LINEAR GRADIENT**

📌 **Definition**

Linear gradient **pattern ki tarah repeat hota hai**

📌 **Syntax**

repeating-linear-gradient(angle/direction, colors);

Examples

```
div {
  background-image: repeating-linear-gradient(
    to right,
```

```
  red 0px,
  red 20px,
  yellow 20px,
  yellow 40px
 );
}
section {
 background-image: repeating-linear-gradient(
  45deg,
  black 0px,
  black 10px,
  white 10px,
  white 20px
 );
}
```

📌 **Use case:** stripes, patterns, lines

---

2 RADIAL GRADIENT

📌 **Definition**

Colors **center se bahar ki taraf gol/oval shape me** spread hote hain

📌 **Syntax**

radial-gradient(shape size at position, colors);

🔷 Shape VALUES

🔶 **circle**

Perfect gol

```
div {
 background-image: radial-gradient(circle, red, yellow);
}
section {
 background-image: radial-gradient(circle, blue, white);
}
```

🔶 **ellipse (default)**

Oval shape

```
div {
 background-image: radial-gradient(ellipse, green, white);
}
section {
 background-image: radial-gradient(ellipse, purple, pink);
```

```
}
```

🔷 **Size VALUES**

🔶 **closest-side**

Nearest side tak

```
div {
  background-image: radial-gradient(circle closest-side, red, yellow);
}
```

🔶 **farthest-corner**

Sabse door corner tak

```
section {
  background-image: radial-gradient(circle farthest-corner, orange, brown);
}
```

🔷 **Position VALUES**

```
div {
  background-image: radial-gradient(circle at center, red, yellow);
}
section {
  background-image: radial-gradient(circle at top left, blue, white);
}
```

🔁 2️⃣ **REPEATING RADIAL GRADIENT**

📌 **Definition**

Radial gradient **rings ki tarah repeat hota hai**

```
div {
  background-image: repeating-radial-gradient(
    circle,
    red 0px,
    red 20px,
    yellow 20px,
    yellow 40px
  );
}
section {
  background-image: repeating-radial-gradient(
    circle,
    black 0px,
    black 10px,
    white 10px,
```

```
  white 20px
 );
}
```

📌 **Use case:** ripple, circular patterns

3️⃣ CONIC GRADIENT

📌 **Definition**

Colors **center ke around circular rotation me** change hote hain

(pizza slice style 🍕)

📌 **Syntax**

conic-gradient(from angle at position, colors);

Examples

```
div {
  background-image: conic-gradient(red, yellow, green);
}
section {
  background-image: conic-gradient(from 90deg, blue, purple, pink);
}
```

🔷 **Percentage Stops**

```
div {
  background-image: conic-gradient(
    red 0%,
    yellow 30%,
    green 60%,
    blue 100%
  );
}
```

🔁 3️⃣ **REPEATING CONIC GRADIENT**

📌 **Definition**

Conic gradient **pie slices ki tarah repeat hota hai**

```
div {
  background-image: repeating-conic-gradient(
    red 0deg,
    red 30deg,
    yellow 30deg,
    yellow 60deg
  );
```

```
}
section {
  background-image: repeating-conic-gradient(
    blue 0deg,
    blue 15deg,
    white 15deg,
    white 30deg
  );
}
```

📌 **Use case:** loaders, charts, wheels

🌫 BOX SHADOWS IN CSS

📌 **box-shadow KYA HOTA HAI?**

👉 box-shadow element ke **bahar ya andar shadow (chhaya)** banata hai

👉 Ye **depth, elevation aur UI feel** dene ke kaam aata hai

👉 Mostly **cards, buttons, modals, panels** me use hota hai

🧱 **BASIC SYNTAX**

```
box-shadow: offset-x offset-y blur-radius spread-radius color inset;
```

⚠️ **Order important hai**

⚠️ inset optional hota hai

🔷 **VALUES & UNKI DEFINITIONS (ONE BY ONE)**

1️⃣ **offset-x (MANDATORY)**

📌 **Definition**

Shadow **left/right** kitni door hogi

- positive → right side
- negative → left side

```
.box { box-shadow: 10px 0px 0px 0px black; }
.box { box-shadow: -10px 0px 0px 0px black; }
```

2️⃣ **offset-y (MANDATORY)**

📌 **Definition**

Shadow **upar/neeche** kitni door hogi

- positive → neeche

- negative → upar

```
.box { box-shadow: 0px 10px 0px 0px black; }
.box { box-shadow: 0px -10px 0px 0px black; }
```

3️⃣ blur-radius (OPTIONAL but IMPORTANT)

📌 **Definition**

Shadow **kitni soft / blur** hogi

- 0 → sharp shadow
- Value badhao → soft shadow

```
.box { box-shadow: 10px 10px 5px black; }
.box { box-shadow: 10px 10px 20px black; }
```

4️⃣ spread-radius (OPTIONAL)

📌 **Definition**

Shadow ka **size badhata ya ghatata** hai

- positive → shadow failta hai
- negative → shadow sikudta hai

```
.box { box-shadow: 10px 10px 10px 5px black; }
.box { box-shadow: 10px 10px 10px -5px black; }
```

5️⃣ color

📌 **Definition**

Shadow ka **rang**

Allowed formats:

- Color name → black
- HEX → #000000
- RGB → rgb(0,0,0)
- RGBA → rgba(0,0,0,0.3) ✅ (BEST)

```
.box { box-shadow: 10px 10px 10px rgba(0,0,0,0.5); }
.box { box-shadow: 10px 10px 10px #333; }
```

6️⃣ inset (OPTIONAL)

📌 **Definition**

Shadow **element ke andar** banata hai (inner shadow)

```
.box { box-shadow: inset 5px 5px 10px rgba(0,0,0,0.4); }
.box { box-shadow: inset -5px -5px 10px rgba(255,255,255,0.6); }
```

📌 Mostly **input fields, neumorphism** me use hota hai

🔁 MULTIPLE BOX SHADOWS

📌 Definition

Ek element par **multiple shadows** lagana (comma-separated)

```
.box {
  box-shadow:
    0 2px 5px rgba(0,0,0,0.3),
    0 10px 20px rgba(0,0,0,0.2);
}
.box {
  box-shadow:
    inset 2px 2px 5px rgba(0,0,0,0.3),
    inset -2px -2px 5px rgba(255,255,255,0.7);
}
```

🎨 COMMON SHADOW STYLES

🔷 Soft Card Shadow

```
.card {
  box-shadow: 0 8px 20px rgba(0,0,0,0.15);
}
```

🔷 Button Hover Shadow

```
button:hover {
  box-shadow: 0 5px 15px rgba(0,0,0,0.4);
}
```

🔷 Floating Effect

```
.float {
  box-shadow: 0 20px 40px rgba(0,0,0,0.3);
}
```

🔷 Neumorphism (Soft UI)

```
.neu {
  box-shadow:
    10px 10px 20px #babecc,
    -10px -10px 20px #ffffff;
}
```

🔷 Inner Input Shadow

```
input {
```

```
box-shadow: inset 2px 2px 6px rgba(0,0,0,0.3);
}
```

⚠️ **COMMON MISTAKES (EXAM + PRACTICAL)**

❌ text-shadow aur box-shadow confuse karna

❌ Dark shadow without opacity

❌ Bahut zyada blur (UI heavy lagta hai)

❌ Mobile performance ignore karna

🎛️ **FILTERS IN CSS — COMPLETE & DETAILED GUIDE**

📌 **CSS filter KYA HOTA HAI?**

👉 filter CSS property **visual effects** apply karti hai

👉 Mostly **images, videos, backgrounds, icons** par use hoti hai

👉 Photoshop jaise effects **direct CSS me**

📌 filter ka effect **render time par** lagta hai (image change nahi hoti)

🔷 **BASIC SYNTAX**

```
filter: filter-function(value);
Multiple filters ek saath:
filter: blur(5px) brightness(120%) contrast(110%);
```

✅ ALL CSS FILTER FUNCTIONS

1️⃣ **blur()**

📌 **Definition**

Element ko **dhundhla (blur)** kar deta hai

Values

- px → blur ki intensity

```
img { filter: blur(2px); }
img { filter: blur(8px); }
```

📌 Zyada value = zyada blur

2️⃣ **brightness()**

📌 **Definition**

Element ki **roshni (lightness)** control karta hai

Values

- 100% → normal
- <100% → dark
- >100% → bright

```
img { filter: brightness(80%); }
img { filter: brightness(150%); }
```

---

3 contrast()

📌 **Definition**

Light aur dark ke beech ka **difference** badhata / ghatata hai

Values

- 100% → normal
- 0% → grey
- >100% → sharp contrast

```
img { filter: contrast(90%); }
img { filter: contrast(200%); }
```

---

4 grayscale()

📌 **Definition**

Image ko **black & white** banata hai

Values

- 0% → original
- 100% → full grayscale

```
img { filter: grayscale(50%); }
img { filter: grayscale(100%); }
```

📌 Common use: hover effects, galleries

---

5 sepia()

📌 **Definition**

Image ko **old / vintage photo** look deta hai

Values

- 0% → normal
- 100% → full sepia

```
img { filter: sepia(40%); }
img { filter: sepia(100%); }
```

---

6 saturate()

📌 **Definition**

Colors ki **intensity (vividness)** control karta hai

Values

- 100% → normal
- <100% → dull
- >100% → extra colorful

```
img { filter: saturate(70%); }
img { filter: saturate(200%); }
```

---

7 invert()

📌 **Definition**

Colors ko **ulta (negative image)** kar deta hai

Values

- 0% → normal
- 100% → fully inverted

```
img { filter: invert(100%); }
.icon { filter: invert(70%); }
```

📌 Dark mode icons ke liye useful

---

8 opacity()

📌 **Definition**

Element ki **transparency** control karta hai

Values

- 0 → invisible
- 1 / 100% → fully visible

```
img { filter: opacity(50%); }
img { filter: opacity(0.3); }
```

📌 opacity: property jaisa hi effect, par filter chain me use hota hai

---

9 hue-rotate()

📌 **Definition**

Image ke colors ka **color wheel angle rotate** karta hai

Values

- deg (0–360)

```
img { filter: hue-rotate(90deg); }
img { filter: hue-rotate(180deg); }
```

📌 Fun & creative effects ke liye

🔟 drop-shadow()

📌 **Definition**

Image ke **actual shape ke according shadow** deta hai

(transparent areas ko ignore karta hai)

Syntax

```
filter: drop-shadow(offset-x offset-y blur color);
img { filter: drop-shadow(5px 5px 10px black); }
.icon { filter: drop-shadow(0 0 15px rgba(0,0,0,0.6)); }
```

📌 PNG / SVG icons ke liye best

📌 box-shadow se different

🔁 MULTIPLE FILTERS EK SAATH

📌 **Definition**

Ek element par **multiple effects** lagana

```
img {
  filter: grayscale(100%) brightness(120%) contrast(110%);
}
img:hover {
  filter: none;
}
```

🎨 COMMON REAL-LIFE USE CASES

🔷 Image Hover Effect

```
img {
  filter: grayscale(100%);
  transition: 0.4s;
}
```

```
img:hover {
  filter: grayscale(0%);
}
```

🔷 Dark Mode Icon

```
.icon {
  filter: invert(100%);
}
```

🔷 Glass / Blur Background

```
.bg {
  filter: blur(10px);
}
```

🔷 Disabled Image Look

```
img.disabled {
  filter: grayscale(100%) opacity(50%);
}
```

❌ COMMON MISTAKES (VERY IMPORTANT)

❌ filter aur box-shadow ko same samajhna
❌ High blur on large images (performance issue)
❌ Mobile devices par heavy filters
❌ drop-shadow aur box-shadow confuse karna

📌 IMPORTANT NOTES

- filter **box model size change nahi karta**
- Animation possible hai with transition
- Mostly **images, videos, SVG** ke liye
- Parent filter children par bhi apply hota hai

🧠 EXAM ONE-LINER

CSS filter property elements par visual effects jaise blur, brightness, grayscale, contrast, shadow apply karti hai.

🏁 FINAL CHECKLIST (SAB COVER)

✅ blur
✅ brightness
✅ contrast
✅ grayscale
✅ sepia
✅ saturate
✅ invert
✅ opacity
✅ hue-rotate
✅ drop-shadow

✅ multiple filters
✅ real use-cases

UNTOLD EDUCATION
Untold skills for modern jobs

📋 LISTS IN CSS —

📌 CSS LIST KYA HOTI HAI?

HTML me list mainly 3 types ki hoti hain:

1️⃣ <ul> → Unordered List (bullets)
2️⃣ <ol> → Ordered List (numbers)
3️⃣ <li> → List Item

CSS se hum:

- bullet / number change karte hain
- position control karte hain
- image / icon lagate hain
- custom marker banate hain

🔷 MAIN LIST PROPERTIES

1️⃣ list-style-type

📌 Definition

List ke **marker ka type** decide karta hai
(bullet ka shape ya numbering style)

🔷 VALUES (WITH DEFINITIONS & EXAMPLES)

🔶 For ul (unordered list)

disc (default)

Solid round bullet

```
ul { list-style-type: disc; }
ul.menu { list-style-type: disc; }
```

---

circle

Hollow round bullet

```
ul { list-style-type: circle; }
ul.nav { list-style-type: circle; }
```

CSS Complete Notes
Styling modern websites from basics to advanced
Created by Pawan Soni
Computer Teacher • Rewari, Haryana

square

Square bullet

```
ul { list-style-type: square; }
ul.box { list-style-type: square; }
```

none

Bullet remove kar deta hai ✅ (MOST USED)

```
ul { list-style-type: none; }
ul.menu { list-style-type: none; }
```

📌 Menus / navbar ke liye must

🔶 **For ol (ordered list)**

decimal

Normal numbers (1,2,3...)

```
ol { list-style-type: decimal; }
ol.num { list-style-type: decimal; }
```

decimal-leading-zero

01, 02, 03...

```
ol { list-style-type: decimal-leading-zero; }
ol.steps { list-style-type: decimal-leading-zero; }
```

lower-alpha

a, b, c...

```
ol { list-style-type: lower-alpha; }
ol.alpha { list-style-type: lower-alpha; }
```

upper-alpha

A, B, C...

```
ol { list-style-type: upper-alpha; }
ol.alpha2 { list-style-type: upper-alpha; }
```

lower-roman

i, ii, iii...

```
ol { list-style-type: lower-roman; }
ol.roman { list-style-type: lower-roman; }
```

upper-roman

I, II, III...

```
ol { list-style-type: upper-roman; }
ol.roman2 { list-style-type: upper-roman; }
```

---

2 list-style-position

📌 **Definition**

Marker **content ke andar hoga ya bahar**, ye decide karta hai

---

🔷 **VALUES**

outside (default)

Bullet content ke **bahar**

```
ul { list-style-position: outside; }
ol { list-style-position: outside; }
```

---

inside

Bullet content ke **andar** (text ke saath align)

```
ul { list-style-position: inside; }
ol { list-style-position: inside; }
```

📌 Long text me difference clearly dikhta hai

---

3 list-style-image

📌 **Definition**

Bullet ki jagah **image / icon** use karta hai

🔷 **VALUES**

url()

Custom image marker

```
ul {
  list-style-image: url("arrow.png");
}
ul.icons {
  list-style-image: url("check.png");
}
```

⚠️ Control kam hota hai → modern CSS me kam use hota hai

---

none

Image marker remove

```
ul {
  list-style-image: none;
}
ul.clean {
  list-style-image: none;
}
```

4️⃣ list-style (SHORTHAND)

📌 **Definition**

type + position + image ek line me

Examples

```
ul {
  list-style: square inside;
}
ol {
  list-style: upper-roman outside;
}
ul {
  list-style: none;
}
```

⭐ MODERN & IMPORTANT PART (MARKER CONTROL)

5️⃣ ::marker (REAL MARKER STYLING)

📌 **Definition**

List ke **actual bullet/number** ko style karta hai

(HTML ke default marker ko hi)

Allowed properties (limited):

- color
- font-size
- content

Examples

```
li::marker {
  color: red;
}
li::marker {
  font-size: 20px;
  color: blue;
}
ol li::marker {
  content: "👉 ";
}
```

📌 Exam + modern CSS ke liye VERY IMPORTANT

🔥 **CUSTOM MARKER (BEFORE METHOD)**

6️⃣ **::before (CUSTOM MARKER BANANA)**

📌 **Definition**

Default marker hata ke **apna marker banana**

Steps:

1. Bullet hatao
2. ::before se icon/text lagao

Example: Tick Mark List ✅

```
ul {
  list-style: none;
}

li::before {
  content: "✓";
  color: green;
}
```

Example: Arrow Marker

```
ul {
  list-style: none;
}
```

```
li::before {
  content: "➜";
  color: orange;
}
```

Example: Font Awesome Icon (advanced)

```
li::before {
  content: "★";
  color: gold;
}
```

📌 **Menus, features list, UI cards** me sabse zyada use hota hai

---

🔷 LIST SPACING CONTROL

7️⃣ margin & padding with Lists

Default browser padding remove karna (EXAM FAVORITE)

```
ul {
  margin: 0;
  padding: 0;
}
ol {
  padding-left: 20px;
}
```

8️⃣ line-height (list item spacing)

```
li {
  line-height: 2;
}
li {
  line-height: 1.5;
}
```

🧠 REAL-LIFE USE CASES

🔷 Navigation Menu

```
ul {
  list-style: none;
```

```
 display: flex;
 gap: 20px;
}
```

🔷 Feature List

```
li::before {
 content: "✓";
 color: green;
}
```

🔷 Steps / Process

```
ol {
 list-style-type: decimal-leading-zero;
}
```

❌ COMMON MISTAKES

❌ list-style-image pe zyada depend karna
❌ ::marker aur ::before confuse karna
❌ Default padding remove na karna
❌ Marker alignment ignore karna

📌 EXAM ONE-LINER

CSS list properties list ke bullets aur numbering ko style aur control karne ke liye use hoti hain.

🏁 FINAL CHECKLIST (SAB COVER)

✅ list-style-type
✅ list-style-position
✅ list-style-image
✅ list-style (shorthand)
✅ ::marker
✅ ::before custom marker
✅ spacing & alignment
✅ real UI usage

Complete Notes
Created by Pawan Soni
Computer Teacher • Rewari, Haryana

🔗 ANCHOR STATES IN CSS —

📌 **ANCHOR STATE KYA HOTA HAI?**

👉 Anchor (<a>) ke **different conditions / states** hote hain

👉 User jab link ke saath interact karta hai, tab state change hoti hai

👉 CSS se hum **har state ka alag style** de sakte hain

🔷 **ANCHOR KE MAIN STATES (EXAM GOLD)**

CSS me **5 main anchor states** hote hain:

1️⃣ a:link

2️⃣ a:visited

3️⃣ a:hover

4️⃣ a:active

5️⃣ a:focus

1️⃣ **a:link**

📌 **Definition**

👉 **Unvisited link** (jis par user ne abhi tak click nahi kiya)

Examples (alag values)

```
a:link {
  color: blue;
}
a:link {
  color: green;
  text-decoration: none;
}
```

📌 Sirf **new / fresh links** par apply hota hai

2️⃣ **a:visited**

📌 **Definition**

👉 **Visited link** (jis par user pehle click kar chuka hai)

Examples

```
a:visited {
```

```
  color: purple;
}
a:visited {
  color: brown;
}
```

⚠️ **Security reason** se limited properties allowed hoti hain (jaise: color, background-color)

3️⃣ a:hover ⭐ (MOST USED)

📌 **Definition**

👉 Jab **mouse link ke upar le jaate hain**

Examples

```
a:hover {
  color: red;
}
a:hover {
  background-color: yellow;
  text-decoration: underline;
}
```

📌 Buttons, menus, navbars ke liye sabse important

4️⃣ a:active

📌 **Definition**

👉 Jab link **click ho raha hota hai** (mouse button pressed)

Examples

```
a:active {
  color: orange;
}
a:active {
  background-color: black;
  color: white;
}
```

📌 Ye state **bahut short time** ke liye hoti hai

UNTOLD EDUCATION
Untold skills for modern jobs

5️⃣ a:focus

📌 **Definition**

👉 Jab link **keyboard (Tab key)** se select hota hai

👉 Accessibility ke liye **bahut important**

Examples

```
a:focus {
  outline: 2px solid red;
}
a:focus {
  background-color: lightblue;
}
```

📌 Screen readers & keyboard users ke liye zaroori

Styling modern websites from basics to advanced

🔥 LVHAF RULE (ORDER VERY IMPORTANT)

Anchor states **isi order me likhne chahiye**, warna kaam nahi karega:

L → V → H → A → F

link → visited → hover → active → focus

✅ **Correct Order Example**

```
a:link    { color: blue; }
a:visited { color: purple; }
a:hover   { color: red; }
a:active  { color: orange; }
a:focus   { outline: 2px solid black; }
```

❌ Galat order me likhne par hover/active kaam nahi karta

🎨 REAL-LIFE STYLING EXAMPLES

🔷 Simple Clean Link (no underline)

```
a {
  text-decoration: none;
  color: #333;
}

a:hover {
  color: #007bff;
```

```
}
```

UNTOLD EDUCATION
Untold skills for modern jobs

🔷 Button-Like Anchor

```
a {
  padding: 10px 20px;
  background: blue;
  color: white;
  text-decoration: none;
}

a:hover {
  background: darkblue;
}

a:active {
  background: black;
}
```

🔷 Navbar Links

```
nav a {
  color: white;
  padding: 10px;
}

nav a:hover {
  background: orange;
}
```

🔷 Accessible Focus Style (BEST PRACTICE)

```
a:focus {
  outline: 3px dashed orange;
}
```

Created by Pawan Soni
Computer Teacher · Rewari, Haryana

❌ COMMON MISTAKES (EXAM + PRACTICAL)

❌ Anchor states ka wrong order

❌ :focus ignore kar dena

❌ Hover ke bina visual feedback

❌ a:visited par restricted properties use karna

📌 **EXAM ONE-LINER**

Anchor states CSS pseudo-classes hain jo link ke different user interaction states (link, visited, hover, active, focus) ko style karne ke liye use hoti hain.

UNTOLD EDUCATION
Untold skills for modern jobs

CSS
Complete Notes
Styling modern websites from basics to advanced

Created by Pawan Soni
Computer Teacher • Rewari, Haryana

🔗 COMBINATORS IN CSS —

📌 **CSS COMBINATOR KYA HOTA HAI?**

👉 **Combinators selectors ke beech ka relation batate hain**

👉 Ye batate hain ki **kaun-sa element kis element se related hai**

Example:

```
div p { color: red; }
```

➡️ div aur p ke beech ka relation = **descendant**

🔷 **CSS ME TOTAL 4 MAIN COMBINATORS HOTE HAIN**

1️⃣ **Descendant Combinator (space)**

2️⃣ **Child Combinator (>)**

3️⃣ **Adjacent Sibling Combinator (+)**

4️⃣ **General Sibling Combinator (~)**

1️⃣ **DESCENDANT COMBINATOR (space)**

📌 **Definition**

👉 Parent ke **andar ke sab child elements** ko select karta hai

👉 Direct ho ya deep level — **sab cover**

📌 **Syntax**

```
parent descendant { }
```

Examples

```
div p {
  color: red;
}
➡️ div ke andar kahin bhi jo p hoga, sab red
section span {
  font-weight: bold;
}
```

📌 **Most commonly used**

2️⃣ **CHILD COMBINATOR (>)**

📌 **Definition**

👉 Sirf **direct child** ko select karta hai

👉 Grandchild ko select **nahi** karta

📌 **Syntax**

```
parent > child { }
```

Examples

```
div > p {
  color: blue;
}
```

➡️ Sirf jo p **directly div ke andar** hai

```
ul > li {
  list-style: none;
}
```

📌 **Strict selection ke liye**

3️⃣ ADJACENT SIBLING COMBINATOR (+)

📌 **Definition**

👉 Sirf **turant next sibling** ko select karta hai

👉 Dono ka **same parent** hona zaroori

📌 **Syntax**

element + sibling { }

Examples

```
h1 + p {
  color: green;
}
➡️ h1 ke bilkul baad wala p
label + input {
  border: 2px solid red;
}
```

📌 **First sibling only**

4️⃣ GENERAL SIBLING COMBINATOR (~)

📌 **Definition**

👉 Same parent ke **saare next siblings** ko select karta hai

👉 Adjacent ho ya door — **sab**

📌 **Syntax**

element ~ sibling { }

Examples

```
h1 ~ p {
  color: orange;
}
/* h1 ke baad aane wale saare p
input:checked ~ label {
  color: red;
}
```

📌 **Checkbox hacks / toggle effects** me bahut use hota hai

🧠 **DIFFERENCE TABLE (EXAM GOLD)**

Combinator	Symbol	Select karta hai
Descendant	space	Sab andar ke
Child	>	Sirf direct
Adjacent Sibling	+	Sirf next
General Sibling	~	Saare next

🔥 **REAL-LIFE USE CASES**

🔷 **Menu Styling**

```
nav ul > li {
  display: inline-block;
}
```

🔷 **Article Paragraphs**

```
article p {
  line-height: 1.8;
}
```

🔷 **Heading ke baad first paragraph**

```
h2 + p {
  font-style: italic;
```

```
}
```

🔷 **Toggle / Checkbox trick**

```
input:checked ~ .content {
  display: block;
}
```

❌ **COMMON MISTAKES**

❌ Descendant aur child ko same samajhna

❌ + se saare siblings expect karna

❌ Parent–child structure samjhe bina selector likhna

📦 DISPLAY IN CSS — s

📌 **display KYA HOTA HAI?**

👉 display CSS property decide karti hai ki

HTML element page par kaise dikhai dega aur kaise behave karega

👉 Ye control karti hai:

- element **line break lega ya nahi**
- **width / height kaam karegi ya nahi**
- **layout system** (flex, grid)

🔷 **MOST IMPORTANT DISPLAY VALUES**

1️⃣ **display: block**

📌 **Definition**

- Element **poori width** leta hai
- **New line se start** hota hai
- width / height kaam karti hai

Common block elements:

div, p, h1–h6, section

Examples

```
div {
  display: block;
}
```

```
span {
  display: block;
  background: yellow;
}
```

📌 Ek block ke baad **next element new line me aata hai**

2️⃣ display: inline

📌 **Definition**

- Element **sirf content jitni width** leta hai
- **Same line me rehta hai**
- width / height ❌ kaam nahi karti

Common inline elements:

span, a, strong

Examples

```
span {
  display: inline;
}
div {
  display: inline;
}
```

📌 Padding left/right kaam karta hai

📌 Padding top/bottom visual hai, layout affect nahi karta

3️⃣ display: inline-block ⭐ (VERY IMPORTANT)

📌 **Definition**

👉 Inline + Block ka **combo**

- Same line me rehta hai ✅
- width / height kaam karti hai ✅

Examples

```
span {
  display: inline-block;
  width: 150px;
  height: 50px;
}
a {
  display: inline-block;
```

```
  padding: 10px 20px;
}
```

📌 Buttons, menus, cards ke liye BEST

4️⃣ **display: none**

📌 **Definition**

👉 Element **poori tarah remove** ho jata hai

👉 **Space bhi nahi rehta**

Examples

```
div {
  display: none;
}
.menu {
  display: none;
}
```

📌 JS / toggle / dropdowns me bahut use hota hai

⚠️ **display: none vs visibility: hidden**

Point	display:none	visibility:hidden
Visible	❌	❌
Space	❌	✅
Layout	remove	preserve

🌊 OVERFLOW IN CSS —

📌 **overflow KYA HOTA HAI?**

👉 Jab **content apne container (box) se zyada bada** ho jaata hai,
to overflow decide karta hai **extra content ka kya hoga**:

- dikhega?
- chhup jaayega?
- scroll aayega?

📌 overflow **box model** se directly related hai.

🔷 **BASIC SYNTAX**

UNTOLD EDUCATION
Untold skills for modern jobs

overflow: value;
Ya direction-wise:
overflow-x: value;
overflow-y: value;

1️⃣ overflow (SHORTHAND PROPERTY)

📌 **Definition**

X (horizontal) aur Y (vertical) **dono directions** ka overflow control karta hai.

🔷 VALUES & UNKI DEFINITIONS

🔶 visible (DEFAULT)

📌 **Definition**

Extra content **box ke bahar bhi dikhega**.
Container ka size change **nahi** hota.

```
div {
  overflow: visible;
}
.box {
  width: 150px;
  height: 80px;
  overflow: visible;
}
```

📌 Default behavior
📌 Layout break ho sakta hai

🔶 hidden

📌 **Definition**

Extra content **chhup jaata hai** (cut ho jaata hai).
Koi scroll bar nahi.

```
div {
  overflow: hidden;
}
.image-box {
  overflow: hidden;
}
```

📌 Image crop, cards, sliders ke liye common

🔶 scroll

📌 **Definition**

Hamesha scroll bar dikhegi, chahe content ho ya na ho.

```
div {
  overflow: scroll;
}
.box {
  width: 200px;
  height: 100px;
  overflow: scroll;
}
```

📌 Vertical + Horizontal dono scroll aa sakte hain

🔶 auto ⭐ (MOST USED)

📌 **Definition**

Scroll bar **sirf tab dikhegi jab zarurat hogi**.

```
div {
  overflow: auto;
}
.content {
  max-height: 150px;
  overflow: auto;
}
```

📌 Best & smart option for most cases

🔶 clip (MODERN)

📌 **Definition**

hidden jaisa hi hai,

lekin **scrolling bilkul allow nahi** karta.

```
div {
  overflow: clip;
}
.card {
  overflow: clip;
}
```

📌 Advanced / modern CSS

📌 clip me scroll position nahi hoti

2️⃣ overflow-x

📌 **Definition**

Sirf **horizontal (left–right)** overflow control karta hai.

Examples

```
div {
  overflow-x: hidden;
}
.container {
  overflow-x: auto;
}
```

📌 Horizontal scroll hide karne ke liye common

3️⃣ overflow-y

📌 **Definition**

Sirf **vertical (top–bottom)** overflow control karta hai.

Examples

```
div {
  overflow-y: scroll;
}
.content {
  overflow-y: auto;
}
```

📌 Chat box, comment box ke liye common

4️⃣ text-overflow (TEXT KE LIYE)

⚠️ Ye overflow ka part hai, par **sirf text** ke liye.

🔷 **text-overflow: clip**

📌 **Definition**

Text **cut ho jaata hai**, koi symbol nahi.

```
p {
```

UNTOLD EDUCATION
Untold skills for modern jobs
Created by Pawan Soni
Computer Teacher • Rewari, Haryana

```
  white-space: nowrap;
  overflow: hidden;
  text-overflow: clip;
}
```

🔷 **text-overflow: ellipsis**

📌 **Definition**

Text cut hoke **... show karta hai**.

```
p {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
```

📌 Cards, titles, product names ke liye VERY IMPORTANT

⚠️ **IMPORTANT RULE (EXAM FAVORITE)**

text-overflow tabhi kaam karega jab:

white-space: nowrap;

overflow: hidden;

dono lage hon.

🔥 **REAL-LIFE USE CASES**

🔷 **Image Crop Card**

```
.card {
  width: 300px;
  height: 200px;
  overflow: hidden;
}
```

🔷 **Scrollable Content Box**

```
.comments {
  max-height: 200px;
  overflow-y: auto;
}
```

🔷 **Horizontal Scroll Table**

```
.table-wrapper {
  overflow-x: auto;
}
```

🔷 **Ellipsis Title**

```
.title {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
```

🧠 **DIFFERENCE TABLE (EXAM GOLD)**

Value	Content dikhega?	Scroll
visible	✅	❌
hidden	❌	❌
scroll	❌	✅ (always)
auto	❌	✅ (if needed)
clip	❌	❌

❌ **COMMON MISTAKES**

❌ Height/width diye bina overflow expect karna

❌ overflow: hidden se content loss bhool jaana

❌ text-overflow bina white-space ke

❌ Mobile par horizontal overflow ignore karna

✨ PSEUDO-ELEMENTS IN CSS —

📌 **PSEUDO-ELEMENT KYA HOTA HAI?**

👉 Pseudo-elements **element ke kisi specific PART** ko style karte hain

👉 HTML me extra tag likhne ki zarurat **nahi hoti**

📌 Example:

```
p::first-letter { color: red; }
```

➡️ Sirf paragraph ka **pehla letter** style hoga

⚠️ **: vs :: (VERY IMPORTANT)**

Type	Use
:hover	pseudo-class (state)
::before	pseudo-element (part)

📌 Modern CSS me **pseudo-elements :: se likhe jaate hain**

📌 Old support ke liye :before bhi kaam karta hai, par **best practice ::**

🔷 **MOST IMPORTANT PSEUDO-ELEMENTS**

1️⃣ **::before ⭐ (MOST USED)**

📌 **Definition**

👉 Element ke **content se pehle** virtual content add karta hai

👉 HTML change nahi hota

⚠️ content property **mandatory** hai

Examples

```
p::before {
  content: "👉 ";
  color: red;
}
h2::before {
  content: "★ ";
  color: gold;
}
```

📌 Use cases:

- icons
- custom list markers
- labels

Created by Pawan Soni
Computer Teacher • Rewari, Haryana

2️⃣ **::after ⭐**

📌 **Definition**

UNTOLD EDUCATION
Untold skills for modern jobs

👉 Element ke **content ke baad** content add karta hai

Examples

```
p::after {
  content: " ✓";
  color: green;
}
a::after {
  content: " ↗";
}
```

📌 Use cases:

- badges
- arrows
- decorative icons

Complete Notes
Styling modern websites from basics to advanced

---

3 ::first-letter

📌 **Definition**

👉 Text ka **pehla letter** select karta hai

Examples

```
p::first-letter {
  font-size: 40px;
  color: red;
}
article::first-letter {
  font-weight: bold;
}
```

📌 Use cases:

- magazine style text
- drop-cap effect

---

4 ::first-line

📌 **Definition**

Created by Pawan Soni
Rewari, Haryana

👉 Text ki **sirf pehli line** select karta hai
(Line responsive hoti hai, screen size se change hoti hai)

Examples

```
p::first-line {
  color: blue;
}
article::first-line {
  font-weight: bold;
}
```

📌 Use cases:

- article highlights
- intro emphasis

5️⃣ ::selection

📌 **Definition**

👉 Jab user text **select karta hai (drag)**, tab ka style

Examples

```
::selection {
  background: yellow;
  color: black;
}
p::selection {
  background: black;
  color: white;
}
```

📌 Allowed properties limited:

- color
- background
- text-shadow

6️⃣ ::marker ⭐ (LIST SPECIAL)

📌 **Definition**

👉 List ke **actual bullet / number** ko style karta hai

Examples

```
li::marker {
  color: red;
}
```

```
ol li::marker {
  font-size: 20px;
  content: "✓";
}
```

📌 Use cases:

- modern list styling
- numbering customization

7️⃣ ::placeholder

📌 **Definition**

👉 Input / textarea ke **placeholder text** ko style karta hai

---

Examples

```
input::placeholder {
  color: gray;
}
textarea::placeholder {
  font-style: italic;
}
```

📌 Forms ke liye VERY IMPORTANT

8️⃣ ::backdrop (ADVANCED)

📌 **Definition**

👉 Fullscreen element ke **background layer** ko style karta hai (modal, dialog)

Example

```
dialog::backdrop {
  background: rgba(0,0,0,0.6);
}
```

📌 Use cases:

- modal overlay
- popup background

🔥 **COMBINATION EXAMPLES**

🔷 Custom Button Icon

```
button::before {
  content: "🚀";
}
```

🔷 Custom List (without bullets)

```
ul {
  list-style: none;
}

li::before {
  content: "✓";
  color: green;
}
```

🔷 Badge on Card

```
.card::after {
  content: "NEW";
  color: white;
  background: red;
  padding: 2px 6px;
}
```

🧠 EXAM DIFFERENCE (IMPORTANT)

Pseudo-Class	Pseudo-Element
:hover	::before
State based	Part based
Interaction	Structure

❌ COMMON MISTAKES

❌ ::before / ::after me content bhool jaana

❌ Pseudo-class aur pseudo-element confuse karna

❌ ::selection me unsupported properties

❌ HTML element samajh lena pseudo-element ko

Created by Pawan Soni
Rewari, Haryana

🎯 PSEUDO-CLASSES IN CSS —

📌 **PSEUDO-CLASS KYA HOTA HAI?**

👉 Pseudo-class **element ki state/condition** ko target karti hai

👉 HTML change kiye bina **interaction ya structure ke base par style** lagta hai

```
a:hover { color: red; }
```

➡️ Jab mouse link par aata hai, tab style apply hota hai

⚠️ **Pseudo-Class vs Pseudo-Element (Quick)**

- **Pseudo-class** → :hover (state)
- **Pseudo-element** → ::before (part)

🔷 **PSEUDO-CLASSES KI MAIN CATEGORIES**

1️⃣ **Link / Interaction**

2️⃣ **Form / Input**

3️⃣ **Structural (Child/Type based)**

4️⃣ **User Action / UI State**

5️⃣ **Logical / Utility**

1️⃣ **LINK & INTERACTION PSEUDO-CLASSES**

🔶 **:link**

Definition: Unvisited link

```
a:link { color: blue; }
a:link { text-decoration: none; }
```

🔶 **:visited**

Definition: Visited link

⚠️ Limited properties allowed (security)

```
a:visited { color: purple; }
a:visited { background-color: #eee; }
```

🔶 **:hover** ⭐

Definition: Mouse element ke upar

```
a:hover { color: red; }
button:hover { background: black; }
```

🔶 **:active**

Definition: Click ke waqt (pressed)

```
a:active { color: orange; }
button:active { transform: scale(0.95); }
```

🔶 **:focus**

Definition: Keyboard (Tab) ya click se focus

```
a:focus { outline: 2px solid red; }
input:focus { border-color: blue; }
```

📌 Accessibility ke liye **must**

🔥 **LVHAF ORDER (EXAM GOLD)**

:link → :visited → :hover → :active → :focus

2️⃣ **FORM / INPUT PSEUDO-CLASSES**

🔶 **:focus**

(Already covered)

🔶 **:checked**

Definition: Checkbox/Radio checked

```
input:checked { accent-color: green; }
input:checked + label { font-weight: bold; }
```

🔶 **:disabled**

Definition: Disabled input

```
input:disabled { background: #ccc; }
button:disabled { cursor: not-allowed; }
```

🔶 **:enabled**

Definition: Enabled input

```
input:enabled { background: white; }
button:enabled { background: blue; }
```

🔶 **:required**

Definition: Required field

```
input:required { border: 2px solid red; }
textarea:required { background: #fff3f3; }
```

◆ **:optional**

Definition: Optional field

```
input:optional { border-color: green; }
textarea:optional { background: #f0fff0; }
```

◆ **:valid**

Definition: Valid input value

```
input:valid { border-color: green; }
```

◆ **:invalid**

Definition: Invalid input value

```
input:invalid { border-color: red; }
```

---

3 STRUCTURAL PSEUDO-CLASSES (VERY IMPORTANT)

◆ **:first-child**

Definition: Parent ka pehla child

```
li:first-child { color: red; }
p:first-child { font-weight: bold; }
```

◆ **:last-child**

Definition: Parent ka last child

```
li:last-child { color: blue; }
div:last-child { margin-bottom: 0; }
```

◆ **:nth-child(n)**

Definition: Position ke base par child

```
li:nth-child(2) { color: green; }
tr:nth-child(even) { background: #eee; }
```

Common values:

- even, odd
- 3, 2n, 2n+1

◆ **:nth-last-child(n)**

Definition: End se count

```
li:nth-last-child(1) { color: red; }
li:nth-last-child(2) { color: blue; }
```

◆ **:first-of-type**

Definition: Type ka pehla element

```
p:first-of-type { color: red; }
```

◆ **:last-of-type**

Definition: Type ka last element

```
p:last-of-type { color: blue; }
```

◆ **:nth-of-type(n)**

Definition: Same type ka nth element

```
p:nth-of-type(2) { font-weight: bold; }
```

◆ **:only-child**

Definition: Parent ka sirf ek child

```
p:only-child { color: purple; }
```

◆ **:only-of-type**

Definition: Us type ka sirf ek element

```
p:only-of-type { background: yellow; }
```

◆ **:empty**

Definition: Jisme koi content nahi

```
div:empty { display: none; }
```

---

4 USER ACTION / UI STATE

◆ **:target**

Definition: URL fragment (#id) match hone par

```
#section1:target { background: yellow; }
```

📌 Single-page navigation

◆ **:root**

Definition: Document ka root (html)

```
:root { --main: blue; }
```

UNTOLD EDUCATION
skills for modern jobs
CSS
color: #2196f3;
Complete Notes
Styling modern websites from basics to advanced
Rowan Soni
Computer Teacher • Rewari, Haryana

◆ :not(selector)

Definition: Selector ko chhod kar sab

```
p:not(.active) { color: gray; }
button:not(:disabled) { cursor: pointer; }
```

◆ :is() (Modern)

Definition: Multiple selectors ko group karna

```
:is(h1, h2, h3) { color: navy; }
```

◆ :where() (Modern)

Definition: :is() jaisa, par **no specificity**

```
:where(h1, h2, h3) { margin: 0; }
```

🔥 REAL-LIFE USE CASES

🔷 Hover Menu

```
nav a:hover { background: orange; }
```

🔷 Zebra Table

```
tr:nth-child(even) { background: #f5f5f5; }
```

🔷 Form Validation

```
input:invalid { border-color: red; }
input:valid { border-color: green; }
```

🔷 Toggle without JS

```
input:checked ~ .panel { display: block; }
```

❌ COMMON MISTAKES

- ❌ :nth-child vs :nth-of-type confuse karna
- ❌ :focus ignore kar dena
- ❌ Anchor states ka wrong order
- ❌ Structural selectors bina HTML structure samjhe

📰 COLUMN LAYOUT IN CSS —

📌 CSS COLUMN LAYOUT KYA HOTA HAI?

👉 CSS Column Layout ka use **content ko newspaper / magazine style columns** me todne ke liye hota hai

👉 Mostly **text-heavy content** ke liye use hota hai (news, blogs, articles)

📌 Ye **Flex/Grid se alag** hai

- Flex/Grid → layout structure
- Columns → **content flow**

🔷 BASIC SYNTAX

```
.element {
  column-count: 3;
}
Ya shorthand:
.element {
  columns: 3 200px;
}
```

🔷 COLUMN LAYOUT KI ALL PROPERTIES

1️⃣ column-count

📌 **Definition**

👉 Content ko **kitne columns me todna hai**, ye decide karta hai

---

Values

- Integer number (1, 2, 3...)

Examples

```
p {
  column-count: 2;
}
.article {
  column-count: 4;
}
```

📌 Browser content ko automatically distribute karta hai

2️⃣ column-width

📌 **Definition**

👉 Har column ki **minimum width** set karta hai

👉 Browser automatically **number of columns decide** karta hai

---

Values

- Length units (px, em, rem)

Examples

```
p {
  column-width: 200px;
}
.article {
  column-width: 300px;
}
```

📌 Responsive layouts ke liye BEST

3️⃣ **columns (SHORTHAND)**

📌 **Definition**

👉 column-count + column-width ek hi line me

Syntax

```
columns: column-width column-count;
```

(order flexible hota hai)

Examples

```
.article {
  columns: 3 250px;
}
.text {
  columns: 200px 2;
}
```

4️⃣ **column-gap**

📌 **Definition**

👉 Columns ke beech ka **gap / space**

UNTOLD EDUCATION
Created by Pawan Soni
Computer Teacher • Rewari, Haryana

Values

- Length (px, em)
- normal (default)

Examples

```
p {
  column-gap: 30px;
}
.article {
  column-gap: 3em;
}
```

📌 Readability ke liye bahut important

5️⃣ **column-rule**

📌 **Definition**

👉 Columns ke beech **vertical line** (border jaisi)

Syntax

column-rule: width style color;

Examples

```
p {
  column-rule: 2px solid black;
}
.article {
  column-rule: 3px dashed gray;
}
```

📌 Ye **column-gap ke beech** lagti hai

6️⃣ **column-rule-width**

📌 **Definition**

👉 Column rule ki **motai**

Examples

```
p {
  column-rule-width: 1px;
}
```

```
.article {
  column-rule-width: 4px;
}
```

7️⃣ column-rule-style

📌 **Definition**

👉 Column rule ka **style**

Values (border jaise hi)

- solid
- dashed
- dotted
- double
- none

Examples

```
p {
  column-rule-style: dotted;
}
.article {
  column-rule-style: double;
}
```

8️⃣ column-rule-color

📌 **Definition**

👉 Column rule ka **color**

Examples

```
p {
  column-rule-color: red;
}
.article {
  column-rule-color: blue;
}
```

9️⃣ column-span ⭐ (VERY IMPORTANT)

📌 **Definition**

👉 Kisi element ko **saare columns ke across** spread karta hai

📌 Mostly headings ke liye

Values

- none (default)
- all

Examples

```
h2 {
  column-span: all;
}
.heading {
  column-span: all;
}
```

📌 Sirf **multi-column container ke andar** kaam karta hai

🔟 column-fill

📌 **Definition**

👉 Content columns me **kaise bharega**, ye control karta hai

Values

🔶 **balance (default)**

Columns ko barabar bharta hai

```
.article {
  column-fill: balance;
}
```

🔶 **auto**

Columns sequentially bharta hai

```
.article {
  height: 400px;
  column-fill: auto;
}
```

📌 Height set hone par difference dikhta hai

🧠 **REAL-LIFE USE CASES**

🔷 Newspaper Article

```
.article {
  column-count: 3;
  column-gap: 40px;
}
```

🔷 Responsive Text Layout

```
.content {
  column-width: 250px;
}
```

🔷 Heading Across Columns

```
h1 {
  column-span: all;
}
```

🔷 Magazine Style

```
.text {
  columns: 2 300px;
  column-rule: 2px solid #ccc;
}
```

⚠️ IMPORTANT NOTES

- Columns sirf **content flow** ke liye hote hain
- Items individually control nahi hote (Flex/Grid jaisa nahi)
- Images sometimes break ho sakti hain → break-inside use karo

```
img {
  break-inside: avoid;
}
```

❌ COMMON MISTAKES

❌ Column layout ko grid samajhna

❌ column-span ko har jagah use karna

❌ Small screen par zyada columns

❌ column-gap ignore karna

🌸 FLEXBOX IN CSS —

📌 **FLEXBOX KYA HOTA HAI?**

👉 Flexbox ek **1-D layout system** hai

👉 Ye elements ko **row ya column** me arrange karta hai

👉 Alignment, spacing, centering **bahut easy** ho jaata hai

📌 Flexbox ka kaam:

- horizontal **ya** vertical layout
- responsive design
- equal height cards
- center alignment (sabse easy)

📐 **FLEXBOX KA STRUCTURE**

```
.parent {
  display: flex;
}
```

- **Flex Container** → parent element
- **Flex Items** → container ke direct children

🔷 **FLEX CONTAINER PROPERTIES (PARENT)**

1️⃣ **display: flex**

📌 **Definition**

Element ko **flex container** banata hai

```
.container {
  display: flex;
}
.nav {
  display: flex;
}
```

2️⃣ **flex-direction**

📌 **Definition**

Flex items **kis direction me** arrange honge

UNTOLD EDUCATION
Untold skills for modern jobs
CSS Complete Notes
Styling modern websites from basics to advanced
Created by Pawan Soni
Computer Teacher • Rewari, Haryana

🔷 **Values & Definitions**

🔶 **row (default)**

Left → Right

```
.container { flex-direction: row; }
```

🔶 **row-reverse**

Right → Left

```
.container { flex-direction: row-reverse; }
```

🔶 **column**

Top → Bottom

```
.container { flex-direction: column; }
```

🔶 **column-reverse**

Bottom → Top

```
.container { flex-direction: column-reverse; }
```

3️⃣ **flex-wrap**

📌 **Definition**

Items **next line me jaayenge ya nahi**

🔷 **Values**

🔶 **nowrap (default)**

Sab ek hi line me

```
.container { flex-wrap: nowrap; }
```

🔶 **wrap**

Next line me wrap

```
.container { flex-wrap: wrap; }
```

🔶 **wrap-reverse**

Reverse direction me wrap

```
.container { flex-wrap: wrap-reverse; }
```

4️⃣ **flex-flow (SHORTHAND)**

📌 **Definition**

flex-direction + flex-wrap

```
.container {
  flex-flow: row wrap;
}
.container {
```

```
 flex-flow: column nowrap;
}
```

5️⃣ justify-content ⭐ (MAIN AXIS)

📌 **Definition**

Items ko **main axis** par align karta hai

🔷 **Values & Definitions**

flex-start (default)

Start se

```
.container { justify-content: flex-start; }
```

flex-end

End par

```
.container { justify-content: flex-end; }
```

center

Center me

```
.container { justify-content: center; }
```

space-between

First start, last end

```
.container { justify-content: space-between; }
```

space-around

Equal space around

```
.container { justify-content: space-around; }
```

space-evenly

Equal space everywhere

```
.container { justify-content: space-evenly; }
```

6️⃣ align-items ⭐ (CROSS AXIS)

📌 **Definition**

Items ko **cross axis** par align karta hai

🔷 **Values**

stretch (default)

Height fill kar leta hai

```
.container { align-items: stretch; }
```

flex-start

Top se

```
.container { align-items: flex-start; }
```

flex-end

Bottom se

```
.container { align-items: flex-end; }
```

center

Vertical center

```
.container { align-items: center; }
```

baseline

Text baseline ke hisaab se

```
.container { align-items: baseline; }
```

---

7 align-content (MULTI-LINE ONLY)

📌 **Definition**

Multiple lines ke beech alignment

⚠️ Sirf jab flex-wrap: wrap ho

🔷 **Values**

```
.container { align-content: flex-start; }
.container { align-content: center; }
.container { align-content: space-between; }
```

---

8 gap (MODERN & IMPORTANT)

📌 **Definition**

Flex items ke beech **space**

```
.container { gap: 20px; }
.container { row-gap: 10px; column-gap: 30px; }
```

📌 Margin ka clean replacement

🔶 FLEX ITEM PROPERTIES (CHILD)

---

9 flex-grow

📌 **Definition**

Item **extra space** kitna lega

```
.item { flex-grow: 1; }
.item2 { flex-grow: 2; }
```

UNTOLD EDUCATION
Untold skills for modern jobs

10 flex-shrink

📌 **Definition**

Item **kitna shrink** hoga

```
.item { flex-shrink: 1; }
.item { flex-shrink: 0; }
```

---

11 flex-basis

📌 **Definition**

Item ka **initial size**

```
.item { flex-basis: 200px; }
.item { flex-basis: 30%; }
```

---

12 flex (SHORTHAND) ⭐

📌 **Definition**

flex-grow flex-shrink flex-basis

```
.item { flex: 1; }
.item { flex: 1 0 200px; }
```

---

13 align-self

📌 **Definition**

Single item ka alignment override

.item { align-self: center; }

.item { align-self: flex-end; }

---

14 order

📌 **Definition**

Item ka **display order change**

```
.item1 { order: 2; }
.item2 { order: -1; }
```

📌 HTML change ki zarurat nahi

🔥 **MOST IMPORTANT CONCEPT**

📐 **MAIN AXIS vs CROSS AXIS**

flex-direction	Main Axis	Cross Axis
row	horizontal	vertical

Created by Pawan Soni
Computer Teacher • Rewari, Haryana

flex-direction	Main Axis	Cross Axis
column	vertical	horizontal

🧠 REAL-LIFE FLEXBOX USE CASES

🔷 Perfect Center (EXAM FAVORITE)

```
.container {
  display: flex;
  justify-content: center;
  align-items: center;
}
```

🔷 Navbar

```
nav {
  display: flex;
  justify-content: space-between;
}
```

🔷 Cards Layout

```
.cards {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
}
```

```
🏷 Equal Height Columns
.columns {
  display: flex;
}
```

❌ COMMON MISTAKES

❌ align-content ko single line me use karna

❌ Main vs cross axis confuse karna

❌ gap ke bajay margins

❌ flex: 1 ka matlab na samajhna

📦 GRID IN CSS —

📌 **CSS GRID KYA HOTA HAI?**

👉 CSS Grid ek **2-Dimensional layout system** hai

👉 Isme **rows + columns dono** control hote hain

👉 Complex layouts (website, dashboard, admin panel) ke liye best

📌 Difference:

- **Flexbox** → 1D (row *ya* column)
- **Grid** → 2D (row *aur* column)

🧩 **GRID STRUCTURE**

```
.container {
  display: grid;
}
```

- **Grid Container** → parent
- **Grid Items** → direct children

🔷 **GRID CONTAINER PROPERTIES (PARENT)**

1️⃣ **display: grid**

📌 **Definition**

Element ko **grid container** bana deta hai

```
.container {
  display: grid;
}
.layout {
  display: grid;
}
```

2️⃣ **grid-template-columns** ⭐

📌 **Definition**

👉 **Columns ka structure** define karta hai

🔷 **Values & Examples**

Fixed units

UNTOLD EDUCATION
Untold skills for modern jobs
CSS Complete Notes
Created by Pawan Soni
Computer Teacher · Rewari, Haryana

```
.container {
 grid-template-columns: 200px 200px 200px;
}
```

Percentage

```
.container {
 grid-template-columns: 30% 40% 30%;
}
```

Fraction (fr) ⭐ (MOST USED)

```
.container {
 grid-template-columns: 1fr 1fr 1fr;
}
.container {
 grid-template-columns: 2fr 1fr;
}
```

📌 fr = available space ka fraction

3️⃣ grid-template-rows

📌 **Definition**

👉 **Rows ka size** set karta hai

Examples

```
.container {
 grid-template-rows: 100px 200px;
}
.container {
 grid-template-rows: 1fr 2fr;
}
```

4️⃣ repeat() ⭐

📌 **Definition**

👉 Same columns/rows ko repeat karne ke liye

Examples

Created by Pawan Soni

```
.container {
 grid-template-columns: repeat(3, 1fr);
}
.container {
```

```
 grid-template-rows: repeat(2, 150px);
}
```

---

5 gap (grid-gap) ⭐

📌 **Definition**

👉 Grid items ke beech ka **space**

Examples

```
.container {
 gap: 20px;
}
.container {
 row-gap: 10px;
 column-gap: 30px;
}
```

---

6 grid-template-areas ⭐ (IMPORTANT)

📌 **Definition**

👉 Grid ko **named areas** me divide karta hai

Example

```
.container {
 grid-template-columns: 1fr 2fr;
 grid-template-areas:
   "header header"
   "sidebar content"
   "footer footer";
}
```

📌 Saath me items par:

```
.header { grid-area: header; }
.sidebar { grid-area: sidebar; }
.content { grid-area: content; }
.footer { grid-area: footer; }
```

📌 **Very clean & readable layout**

---

7 justify-items

📌 **Definition**

👉 Items ko **horizontal axis** par align karta hai (cell ke andar)

Values

- start
- end
- center
- stretch (default)

```
.container {
 justify-items: center;
}
```

8️⃣ **align-items**

📌 **Definition**

👉 Items ko **vertical axis** par align karta hai

Values

- start
- end
- center
- stretch

```
.container {
 align-items: center;
}
```

9️⃣ **place-items (SHORTHAND)**

📌 **Definition**

👉 align-items + justify-items

```
.container {
 place-items: center;
}
```

🔟 **justify-content**

📌 **Definition**

👉 Puri grid ko **container ke andar horizontally** align karta hai
(Extra space hone par)

Values

```
start | end | center | space-between | space-around | space-evenly
.container {
 justify-content: center;
}
```

1 1 align-content

📌 **Definition**

👉 Puri grid ko **vertical direction** me align karta hai

```
.container {
 align-content: center;
}
```

1 2 place-content (SHORTHAND)

```
.container {
 place-content: center;
}
```

🔶 GRID ITEM PROPERTIES (CHILD)

1 3 grid-column-start / end

📌 **Definition**

👉 Item **kaunsa column se kaunsa column tak** jaayega

Examples

```
.item {
 grid-column-start: 1;
 grid-column-end: 3;
}
```

1 4 grid-row-start / end

```
.item {
 grid-row-start: 1;
 grid-row-end: 3;
}
```

1 5 grid-column (SHORTHAND)

```
.item {
 grid-column: 1 / 4;
}
.item {
 grid-column: span 2;
}
```

---

1 6 grid-row (SHORTHAND)

```
.item {
 grid-row: 2 / 4;
}
```

---

1 7 grid-area ⭐

📌 **Definition**

👉 Item ko **area name** assign karta hai
ya row/column shortcut

Examples

```
.item {
 grid-area: header;
}
.item {
 grid-area: 1 / 1 / 3 / 3;
}
```

---

1 8 justify-self

📌 **Definition**

👉 Ek **single item ka horizontal alignment**

```
.item {
 justify-self: end;
}
```

---

1 9 align-self

📌 **Definition**

👉 Ek **single item ka vertical alignment**

```
.item {
  align-self: center;
}
```

2️⃣0️⃣ place-self (SHORTHAND)

```
.item {
  place-self: center;
}
```

🔥 ADVANCED GRID FUNCTIONS

🔷 minmax()

📌 **Definition**

👉 Minimum aur maximum size define karta hai

```
.container {
  grid-template-columns: minmax(200px, 1fr) 1fr;
}
```

🔷 auto-fit & auto-fill

```
.container {
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
}
.container {
  grid-template-columns: repeat(auto-fill, minmax(150px, 1fr));
}
```

📌 Responsive grids ke liye 🔥

🧠 REAL-LIFE GRID USE CASES

🔷 Website Layout

```
.layout {
  display: grid;
  grid-template-columns: 1fr 3fr;
}
```

🔷 Image Gallery

```
.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 20px;
}
```

🔷 **Dashboard**

```
.dashboard {
  display: grid;
  grid-template-areas:
    "nav nav"
    "menu main"
    "footer footer";
}
```

❌ **COMMON MISTAKES**

❌ Grid aur flex confuse karna

❌ fr unit ka matlab na samajhna

❌ align-items vs align-content confuse

❌ Over-complicated grid

🔄 TRANSFORM IN CSS —

📌 **CSS transform KYA HOTA HAI?**

👉 transform CSS property element ka **shape, size, position ya rotation** change karti hai

👉 Ye **layout ko affect nahi karti** (box model same rehta hai)

👉 Mostly **animations, hover effects, UI motion** ke liye use hoti hai

📌 Transform visually change karta hai, **space reserve wahi rehti hai**

🧱 **BASIC SYNTAX**

transform: transform-function(value);

Multiple transforms ek saath:

```
transform: translateX(20px) rotate(45deg) scale(1.2);
```

🔷 TRANSFORM KE MAIN TYPES

1️⃣ **Translate (move)**
2️⃣ **Scale (size)**
3️⃣ **Rotate (ghumana)**
4️⃣ **Skew (tilt)**
5️⃣ **3D Transform**
6️⃣ **Transform Origin & Perspective**

1️⃣ TRANSLATE (MOVE)

📌 Definition

👉 Element ko **X / Y / Z direction me move** karta hai
👉 Original position se shift hota hai

🔷 translateX()

Horizontal move

```
.box { transform: translateX(50px); }
.box { transform: translateX(-30px); }
```

🔷 translateY()

Vertical move

```
.box { transform: translateY(40px); }
.box { transform: translateY(-20px); }
```

🔷 translate()

X + Y ek saath

```
.box { transform: translate(30px, 20px); }
.box { transform: translate(-10px, 50px); }
```

🔷 translateZ() (3D)

Depth me move

```
.box { transform: translateZ(50px); }
```

🔷 translate3d()

X, Y, Z ek saath

.box { transform: translate3d(20px, 30px, 40px); }

2️⃣ SCALE (SIZE)

📌 Definition

👉 Element ka **size bada ya chhota** karta hai

🔷 scale()

Width + Height dono

```
.box { transform: scale(1.2); }
.box { transform: scale(0.8); }
```

🔷 scaleX()

Sirf width

```
.box { transform: scaleX(1.5); }
.box { transform: scaleX(0.5); }
```

🔷 scaleY()

Sirf height

```
.box { transform: scaleY(1.3); }
.box { transform: scaleY(0.6); }
```

🔷 **scaleZ() (3D)**

```
.box { transform: scaleZ(2); }
```

🔷 **scale3d()**

```
.box { transform: scale3d(1.2, 1.2, 1.5); }
```

3️⃣ ROTATE (ROTATION)

📌 **Definition**

👉 Element ko **ghumata (rotate)** hai

🔷 **rotate()**

2D rotation

```
.box { transform: rotate(45deg); }
.box { transform: rotate(-30deg); }
```

🔷 **rotateX() (3D)**

```
.box { transform: rotateX(45deg); }
.box { transform: rotateX(90deg); }
```

🔷 **rotateY() (3D)**

```
.box { transform: rotateY(45deg); }
.box { transform: rotateY(180deg); }
```

🔷 **rotateZ()**

Same as rotate()

```
.box { transform: rotateZ(60deg); }
```

🔷 rotate3d()

```
.box {
 transform: rotate3d(1, 1, 0, 45deg);
}
```

4️⃣ SKEW (TILT)

📌 Definition

👉 Element ko **tircha (tilt)** karta hai

🔷 skewX()

```
.box { transform: skewX(20deg); }
.box { transform: skewX(-15deg); }
```

🔷 skewY()

```
.box { transform: skewY(20deg); }
.box { transform: skewY(-10deg); }
```

🔷 skew()

```
.box { transform: skew(20deg, 10deg); }
```

5️⃣ 3D TRANSFORM PROPERTIES

🔷 perspective

📌 Definition

👉 3D depth ka **distance** set karta hai

👉 Zyada value = kam depth effect

```
.container {
 perspective: 800px;
}
.scene {
 perspective: 1200px;
}
```

🔷 **transform-style**

Values:

- flat (default)
- preserve-3d

```
.parent {
 transform-style: preserve-3d;
}
```

🔷 **backface-visibility**

📌 **Definition**

👉 Element ka **pichhla side dikhe ya nahi**

```
.card {
 backface-visibility: hidden;
}
.card {
 backface-visibility: visible;
}
```

6️⃣ **transform-origin** ⭐

📌 **Definition**

👉 Transform **kis point se apply hoga**, ye decide karta hai

UNTOLD EDUCATION
Untold skills for modern jobs
CSS
Complete Notes

Values

- center (default)
- top left
- bottom right
- %
- px

Examples

```
.box {
  transform-origin: center;
  transform: rotate(45deg);
}
.box {
  transform-origin: top left;
  transform: rotate(30deg);
}
```

🔁 **MULTIPLE TRANSFORMS EK SAATH**

```
.box {
  transform: translateX(20px) rotate(30deg) scale(1.2);
}
.card {
  transform: scale(1.05) rotateY(10deg);
}
```

📌 Order matter karta hai ⚠️

Created by Pawan Soni
Computer Teacher • Rewari, Haryana

🔥 **REAL-LIFE USE CASES**

🔷 **Hover Zoom**

```
.card {
  transition: transform 0.3s;
}

.card:hover {
  transform: scale(1.1);
}
```

🔷 Center Element (EXAM FAVORITE)

```
.box {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
}
```

🔷 3D Card Flip

```
.card:hover {
  transform: rotateY(180deg);
}
```

🔷 Button Press Effect

```
button:active {
  transform: scale(0.95);
}
```

⚠️ IMPORTANT RULES

- transform **layout break nahi karta**
- Performance best with:
  - transform
  - opacity

- display transform nahi hota
- Transform **inline elements** par kaam karta hai (better with inline-block)

❌ COMMON MISTAKES

❌ Margin se animation try karna
❌ Transform origin ignore karna
❌ Order of transform functions galat
❌ Perspective bina parent ke

🎞️ TRANSITIONS IN CSS —

📌 CSS TRANSITION KYA HOTA HAI?

👉 Transition ka matlab:
ek state se doosri state me smooth change

👉 Jab CSS property change hoti hai (hover, focus, class change),
transition us change ko **slow & smooth** bana deta hai.

❌ Animation jaisa complex nahi
✅ Simple & lightweight

🔷 BASIC IDEA (ONE LINE)

before → (transition) → after

🧱 TRANSITION KA BASIC SYNTAX

transition: property duration timing-function delay;

Example:

transition: background-color 0.3s ease 0s;

1️⃣ transition-property

📌 **Definition**

👉 Batata hai **kaunsi CSS property animate hogi**

UNTOLD EDUCATION
Untold skills for modern jobs

🔷 **VALUES & DEFINITIONS**

🔶 **Specific property**

Sirf wahi animate hogi

```
div {
  transition-property: background-color;
}
button {
  transition-property: transform;
}
```

Complete Notes

🔶 **all (MOST USED)**

Sab change hone wali properties animate hongi

```
div {
  transition-property: all;
}
.card {
  transition-property: all;
}
```

⚠️ Performance ke liye kabhi-kabhi avoid

🔶 **Multiple properties**

Comma separated

```
div {
  transition-property: background-color, transform;
}
.box {
  transition-property: width, height;
}
```

2️⃣ **transition-duration** ⭐

📌 **Definition**

👉 Transition **kitne time me complete hogi**

Created by Pawan Soni

🔷 **VALUES**

Computer Teacher · Rewari, Haryana

- s → seconds
- ms → milliseconds

```
div {
```

```
 transition-duration: 0.5s;
}
button {
 transition-duration: 300ms;
}
```

⚠️ Duration na ho to transition **nahi chalegi**

3️⃣ **transition-timing-function**

📌 **Definition**

👉 Transition **kis speed pattern** me chalegi

🔷 **ALL VALUES & DEFINITIONS**

🔶 **ease (default)**

Start slow → fast → end slow

```
div {
 transition-timing-function: ease;
}
```

🔶 **linear**

Same speed poore time

```
div {
 transition-timing-function: linear;
}
```

🔶 **ease-in**

Start slow → end fast

```
div {
 transition-timing-function: ease-in;
}
```

🔶 **ease-out**

Start fast → end slow

```
div {
 transition-timing-function: ease-out;
}
```

🔶 **ease-in-out**

Start slow → fast → end slow (balanced)

```
div {
  transition-timing-function: ease-in-out;
}
```

🔶 **steps(n)**

Jump animation (no smoothness)

```
div {
  transition-timing-function: steps(5);
}
.box {
  transition-timing-function: steps(10);
}
```

📌 Sprite / pixel animations

🔶 **cubic-bezier(x1,y1,x2,y2)** ⭐ **(ADVANCED)**

📌 **Definition**

👉 Custom animation curve

```
div {
  transition-timing-function: cubic-bezier(0.68, -0.55, 0.27, 1.55);
}
.box {
  transition-timing-function: cubic-bezier(0.25, 0.1, 0.25, 1);
}
```

📌 Advanced UI motion design

4️⃣ **transition-delay**

📌 **Definition**

👉 Transition **kitni der baad start hogi**

🔷 **VALUES**

```
div {
  transition-delay: 0.2s;
}
button {
```

```
 transition-delay: 500ms;
}
```

📌 Hover delay, staggered effects

5️⃣ transition (SHORTHAND) ⭐

📌 **Definition**

👉 Saari transition properties **ek line me**

Examples

```
div {
 transition: all 0.3s ease;
}
button {
 transition: background-color 0.4s ease-in-out 0.1s;
}
```

🔁 MULTIPLE TRANSITIONS

📌 **Definition**

👉 Alag-alag properties ko alag timing dena

```
div {
 transition:
  background-color 0.3s ease,
  transform 0.5s ease-in-out;
}
.card {
 transition:
  width 0.4s linear,
  height 0.6s ease;
}
```

🔥 PROPERTIES JO TRANSITION KE SAATH ACHHI CHALTI HAIN

✅ color
✅ background-color
✅ opacity
✅ transform ⭐ (BEST)

Created by Pawan Soni
Computer Teacher • Rewari, Haryana

UNTOLD EDUCATION
Untold skills for modern jobs

✅ box-shadow
✅ border-radius
❌ display (never animates)

🎨 REAL-LIFE EXAMPLES

🔷 Hover Button

```
button {
  background: blue;
  transition: background 0.3s ease;
}

button:hover {
  background: red;
}
```

🔷 Scale Card on Hover

```
.card {
  transition: transform 0.4s ease;
}

.card:hover {
  transform: scale(1.05);
}
```

🔷 Fade Effect

```
.box {
  opacity: 0.5;
  transition: opacity 0.3s;
}

.box:hover {
  opacity: 1;
}
```

🔷 Shadow Smooth Change

```
.card {
```

```
  box-shadow: 0 0 0 rgba(0,0,0,0);
  transition: box-shadow 0.3s;
}

.card:hover {
  box-shadow: 0 10px 30px rgba(0,0,0,0.3);
}
```

⚠️ **IMPORTANT RULES (EXAM FAVORITE)**

1️⃣ Transition **starting state me likhi jaati hai**, hover me nahi

2️⃣ Duration ke bina transition **work nahi karti**

3️⃣ display transition nahi hota

4️⃣ transform & opacity **best performance**

❌ **COMMON MISTAKES**

❌ Transition hover block me likhna

❌ display: none animate karna

❌ Bahut zyada duration

❌ all ka blind use

🎬 ANIMATIONS IN CSS —

📌 **CSS ANIMATION KYA HOTI HAI?**

👉 CSS Animation element ko **automatically move / change** kar sakti hai

👉 Hover ya click ki zarurat **nahi hoti**

👉 Animation **khud se start hoti hai**

📌 Difference:

- **Transition** → 2 states ke beech
- **Animation** → multiple states (0%, 50%, 100%)

📦 **BASIC STRUCTURE (2 PARTS)**

1️⃣ **@keyframes → animation ka blueprint**

2️⃣ **animation-* properties → animation apply karna**

🧩 **BASIC EXAMPLE**

```
@keyframes move {
 from { transform: translateX(0); }
 to   { transform: translateX(200px); }
}

.box {
 animation: move 2s;
}
```

🔷 1️⃣ **@keyframes (MOST IMPORTANT)**

📌 **Definition**

👉 Animation ke **steps / stages** define karta hai

🔷 **Syntax**

```
@keyframes animation-name {
 from { }
 to   { }
}
OR (recommended)
@keyframes animation-name {
 0%   { }
 50%  { }
 100% { }
}
```

🔷 **Examples**

🔶 **Simple Color Change**

```
@keyframes colorChange {
 0%   { background: red; }
 50%  { background: yellow; }
 100% { background: green; }
}
```

🔷 *Move + Rotate*

```
@keyframes moveRotate {
 0%   { transform: translateX(0) rotate(0); }
 100% { transform: translateX(200px) rotate(360deg); }
```

```
}
```

🔶 ANIMATION PROPERTIES (ALL)

2️⃣ animation-name

📌 **Definition**

👉 Kaunsa @keyframes use hoga

```
.box { animation-name: move; }
.card { animation-name: colorChange; }
```

3️⃣ animation-duration ⭐

📌 **Definition**

👉 Animation **kitne time me complete hogi**

Values

- s (seconds)
- ms (milliseconds)

```
.box { animation-duration: 2s; }
.box { animation-duration: 500ms; }
```

⚠️ Duration na ho → animation nahi chalegi

4️⃣ animation-timing-function

📌 **Definition**

👉 Animation **kis speed pattern** me chalegi

🔷 ALL VALUES + DEFINITIONS

Value	Meaning
ease	slow → fast → slow (default)
linear	same speed
ease-in	slow start
ease-out	slow end
ease-in-out	smooth start & end
steps(n)	jump animation
cubic-bezier()	custom curve

Examples

```
.box { animation-timing-function: linear; }
.box { animation-timing-function: ease-in-out; }
.box { animation-timing-function: steps(5); }
```

---

5 animation-delay

📌 **Definition**

👉 Animation **kitni der baad start hogi**

```
.box { animation-delay: 1s; }
.box { animation-delay: 500ms; }
```

📌 Loading animations me useful

---

6 animation-iteration-count

📌 **Definition**

👉 Animation **kitni baar repeat hogi**

Values

Number

```
.box { animation-iteration-count: 3; }
```

infinite ⭐

```
.loader { animation-iteration-count: infinite; }
```

📌 Loader / spinner ke liye MUST

---

7 animation-direction

📌 **Definition**

👉 Animation **kis direction me chalegi**

🔷 **Values + Definitions**

Value	Meaning
normal	start → end
reverse	end → start
alternate	forward then backward
alternate-reverse	backward then forward

Examples

```
.box { animation-direction: reverse; }
.box { animation-direction: alternate; }
```

---

8 animation-fill-mode ⭐ (IMPORTANT)

📌 **Definition**

👉 Animation ke **before / after state** ko control karta hai

🔷 **Values**

none (default)

No effect before/after

```
.box { animation-fill-mode: none; }
```

---

forwards

Animation ke baad **last state hold**

```
.box { animation-fill-mode: forwards; }
```

---

backwards

Start se pehle first keyframe apply

```
.box { animation-fill-mode: backwards; }
```

---

both

Forwards + backwards

```
.box { animation-fill-mode: both; }
```

---

9 animation-play-state

📌 **Definition**

👉 Animation **pause ya play**

Values

```
.box { animation-play-state: running; }
.box { animation-play-state: paused; }
📌 Hover pause effect:
.box:hover {
  animation-play-state: paused;
}
```

---

10 animation (SHORTHAND) ⭐

📌 **Definition**

👉 Saari animation properties **ek line me**

Syntax

animation: name duration timing delay iteration direction fill-mode;

Examples

```
.box {
  animation: move 2s ease 0s infinite alternate;
}
.card {
  animation: fade 1s linear forwards;
}
```

🔁 **MULTIPLE ANIMATIONS**

```
.box {
  animation:
    move 2s ease infinite,
    colorChange 3s linear infinite;
}
```

🎨 **REAL-LIFE ANIMATION EXAMPLES**

🔷 **Loading Spinner**

```
@keyframes spin {
  100% { transform: rotate(360deg); }
}

.loader {
  animation: spin 1s linear infinite;
}
```

🔷 **Fade In**

```
@keyframes fadeIn {
  from { opacity: 0; }
  to   { opacity: 1; }
}
```

```
.box {
  animation: fadeIn 1s ease forwards;
}
```

🔷 Bounce Effect

```
@keyframes bounce {
  0%,100% { transform: translateY(0); }
  50%    { transform: translateY(-50px); }
}

.ball {
  animation: bounce 1s infinite;
}
```

🔷 Slide In

```
@keyframes slide {
  from { transform: translateX(-100%); }
  to   { transform: translateX(0); }
}
```

⚠️ IMPORTANT RULES (EXAM FAVORITE)

1️⃣ @keyframes name aur animation-name same hona chahiye

2️⃣ Duration mandatory hai

3️⃣ transform + opacity best performance

4️⃣ Infinite animations carefully use karo

❌ COMMON MISTAKES

❌ Animation aur transition confuse karna

❌ display animate karna

❌ Bahut zyada animations

❌ Accessibility ignore karna (motion sickness)

📌 Solution:

```
@media (prefers-reduced-motion: reduce) {
  * { animation: none; }
}
```

UNTOLD EDUCATION

Untold skills for modern jobs

CSS

Complete Notes

Styling modern websites from basics to advanced

Created by **Pawan Soni**

Computer Teacher · Rewari, Haryana